Introduction to Machine Learning

Technical Notes

Rodrigo Kang

Table of contents

1 From Data to Models

The maxim commonly attributed to the English philosopher Francis Bacon, ***scientia potentia est*** — *knowledge is power* — provides a useful starting point for understanding both the scope and the limitations of what we now call **Data Science**. Although nothing resembling the modern discipline existed in Bacon's time, his view of knowledge contains the seed of an idea that remains fundamental to Data Science: claims about the world should ultimately be accountable to empirical evidence.

The debate over how human beings acquire knowledge long predates Bacon. It can be traced back to Greek philosophy, particularly Plato and Aristotle, and was later developed by medieval scholasticism through the **problem of universals** and the opposition between realism and nominalism. Bacon, however, placed particular emphasis on experience. He argued that scientific knowledge should be grounded in careful and systematic observation of nature, with **inductive**

reasoning playing a central role in moving from particular observations towards more general claims. Although the specific procedures of the Baconian method did not have a lasting influence, the broader principles behind his approach were highly influential in the development of scientific methodology. For this reason, Bacon is commonly regarded as one of the founders of the scientific method and as the father of empiricism.

This distinction provides a useful conceptual starting point for Data Science. We can separate two closely related components of the process of acquiring knowledge: the **empirical evidence** obtained from observations, represented by data, and the conceptual or mathematical structures through which we interpret that evidence, represented by models.

In this context, a useful working definition is:

A model is an abstract and simplified representation of some aspect of reality.

This definition immediately establishes an important principle: **models must be fitted to the data, not the data to the models**.

Data are not reality itself. They are observations or measurements of reality and may contain noise, measurement error, sampling bias, or other imperfections. Nevertheless, they constitute the empirical evidence against which our models must be evaluated. If a model is inconsistent with the available evidence, we should question, revise, or replace the model rather than manipulate the evidence merely to preserve it.

Moreover, there is generally no single model capable of completely representing a phenomenon. Different models may capture different aspects of the same underlying process, rely on different assumptions, or provide different trade-offs between simplicity, interpretability, and predictive performance. Models should therefore be regarded as provisional representations that can be refined or replaced as new evidence becomes available.

This relationship between data and models is summarised in Figure 1.

This emphasis on empirical evidence does not diminish the importance of constructing models to interpret the data. On the contrary, model construction is an essential part of scientific reasoning. To understand why, it is useful to return to the distinction between **induction** and **deduction**.

Inductive reasoning allows us to move from particular observations towards more general hypotheses. It is therefore crucial for recognising patterns and generating explanations. However, induction has a fundamental limitation: observations can *support* a general conclusion, but they can never *prove* its universal truth with logical certainty.

This difficulty was famously formulated by David Hume in the eighteenth century and is known as the ***problem of induction***. Past observations, however numerous or consistent, do not logically guarantee that the same regularity will continue to hold in the future.

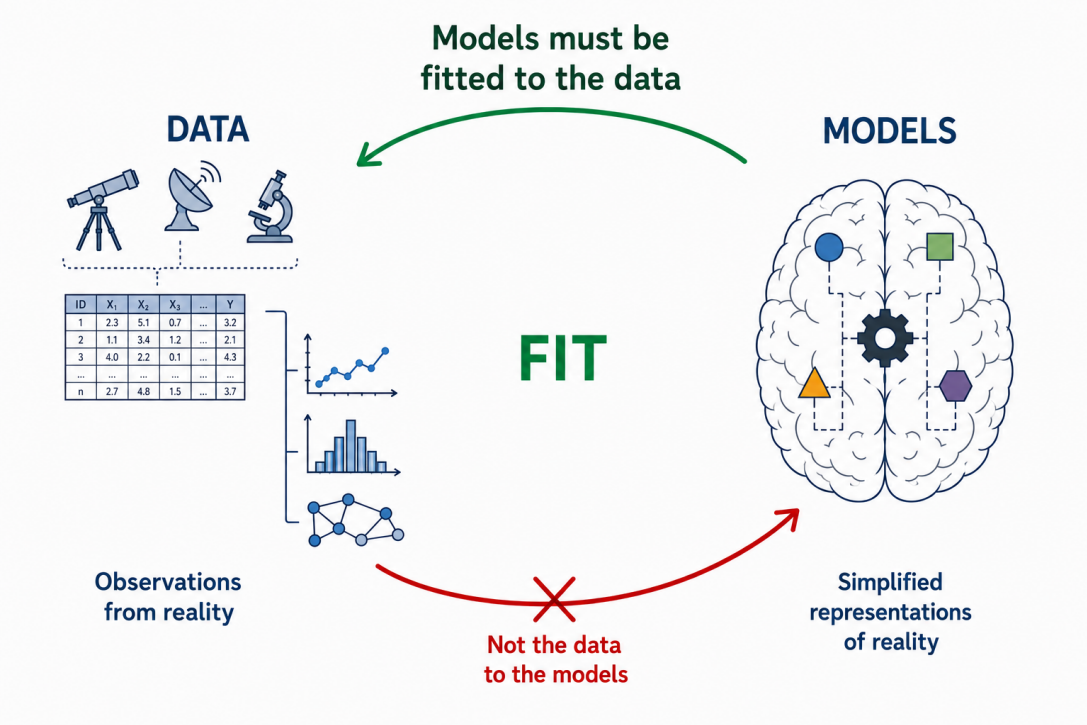

Figure 1: Models are fitted to data: empirical evidence constrains the model, rather than the model determining the evidence.

A well-known illustration of this problem is the ***turkey illusion***, a variation of an example originally introduced by Bertrand Russell using a chicken. Consider a turkey that observes that every morning its owner brings it food. After hundreds of consistent observations — on rainy days and sunny days, in warm weather and cold — the turkey becomes increasingly confident that it will continue to be fed every morning. Yet one day, instead of being fed, it is slaughtered. A single event contradicts a conclusion that appeared to be supported by an overwhelming amount of previous evidence.

The lesson is fundamental: no number of past observations can logically guarantee the truth of a universal claim about the future. The turkey's mistake was not that it relied on observations, but that it extrapolated an observed regularity without understanding the underlying mechanism responsible for it — in this case, the intentions of the person feeding it.

Karl Popper developed this problem further by emphasising **falsifiability**. Induction cannot establish a universal scientific theory as certainly true merely by accumulating confirming observations, whereas evidence inconsistent with its predictions may provide grounds for rejecting or revising it. In practice, scientific testing is more complicated than a single observation mechanically falsifying an entire theory, but the underlying principle remains important: scientific models must expose themselves to empirical testing.

None of this implies that induction is a poor method of reasoning. It serves a different purpose.

Induction remains essential for identifying patterns and generating hypotheses and conjectures. It can be understood in probabilistic rather than absolute terms: observations can increase or decrease the degree of support for a hypothesis without certifying its truth.

Scientific inquiry therefore combines **induction** and **deduction**. Induction helps us move from observations towards hypotheses; deduction allows us to derive testable predictions from those hypotheses and compare them with empirical evidence. Successful predictions provide support for a hypothesis under the conditions tested, but they do not establish it as an absolute truth. Failed predictions may require the hypothesis, its assumptions, or even the experimental procedure to be reconsidered.

For our purposes, the scientific process can be usefully represented through a **hypothetico-deductive cycle**, consisting broadly of the following stages:

- **Systematic observation** of a phenomenon.
- **Formulation of hypotheses or models** to explain the observed patterns.
- **Deduction of testable predictions** from those hypotheses.
- **Empirical evaluation** of the predictions using new observations or data.
- **Revision, refinement, or replacement** of the hypotheses in light of the evidence.

Figure 2 illustrates this iterative process.

The construction of useful models is, of course, considerably more difficult than this simplified representation might suggest. Choosing an appropriate representation, defining an objective, estimating model parameters, evaluating performance, and determining whether the resulting model generalises beyond the observed data introduce a new set of problems.

For this reason, the process of building models deserves separate treatment and is the subject of the next section.

2 Approaches to Modelling

The relationships underlying an observed phenomenon are rarely given to us explicitly. Scientists usually begin with observations obtained from experiments, measurements, or other data-generating processes and attempt to identify patterns and relationships among them.

Mathematics provides a particularly useful language for expressing these relationships in a compact and precise form. The process of constructing an abstract and simplified representation of a phenomenon is what we call **modelling**.

Models are generally built for two closely related purposes:

- **Explanation and understanding:** to describe relationships in the data and improve our understanding of the phenomenon that generated them.

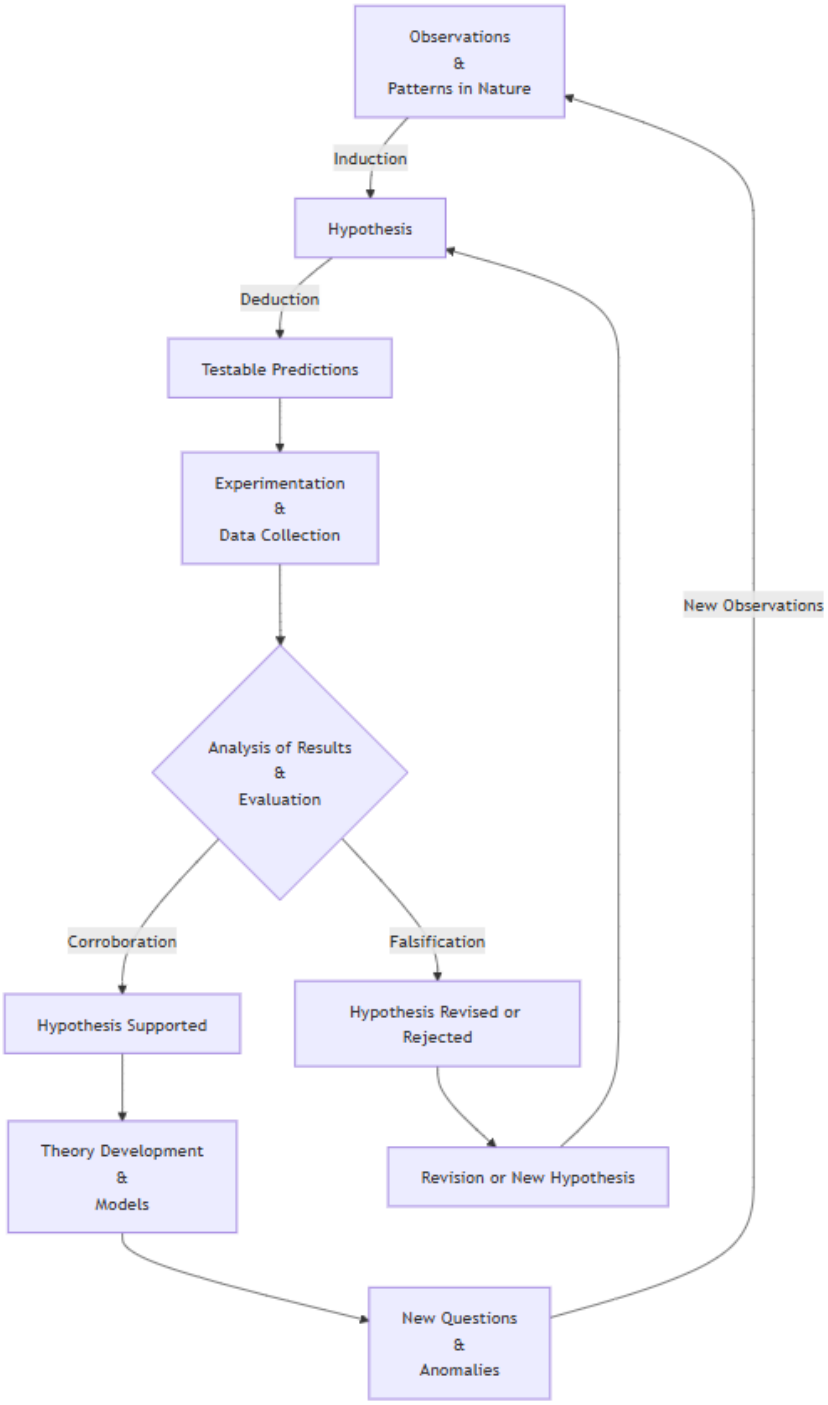

Figure 2: The hypothetico-deductive method. Observations motivate hypotheses through induction; hypotheses generate testable predictions through deduction; empirical results support or challenge those hypotheses, leading to theory development or revision and, ultimately, to new questions that restart the process.

- **Prediction and estimation:** to estimate outcomes for observations that have not yet been seen.

These objectives are related, but they are not the same. A model may provide useful insight into an underlying mechanism without being the best possible predictor. Conversely, a model may achieve strong predictive performance while providing relatively little insight into the mechanism that generated the data.

Prediction also has an important practical role. It allows us to assess whether a model remains useful beyond the observations used to construct it and, in applied settings, to anticipate future or unknown outcomes.

To make these ideas concrete, consider a simple physical experiment.

2.1 A Simple Harmonic Oscillator

Suppose that a spring with spring constant k is fixed to a wall at one end and attached to an object of mass m at the other. For simplicity, assume that the object moves horizontally over an ideal frictionless surface.

We displace the object slightly from its equilibrium position — without stretching the spring beyond its elastic regime — and release it. We observe that the object moves repeatedly backwards and forwards.

This system, illustrated in Figure 3, is known as a **simple harmonic oscillator**.

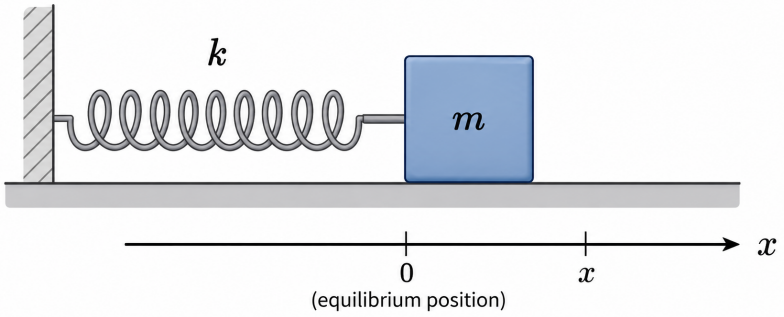

Figure 3: A simple harmonic oscillator consisting of a mass attached to a spring on an ideal frictionless surface.

Suppose that we want to describe the dynamics of the system. We can measure the position x of the object at different times t and record the resulting observations as shown in Table 1.

Position x	Time t
$x_0 = x_{\text{max}}$	$t_0 = 0$
x_2	t_2
$\vdots$	$\vdots$
x_i	t_i
$\vdots$	$\vdots$
x_N	t_N

Table 1: Position of the mass observed at different times.

Using terminology common in statistics and machine learning, we can regard time t as the **predictor**, **covariate**, or **independent variable**, and position x as the **response** or **dependent variable**.

At its simplest, our modelling problem can be written as

$$x = f(t), \tag{1}$$

where f represents the unknown relationship between time and position.

This notation should not be interpreted as implying that modelling problems generally reduce to finding an explicit algebraic relationship directly between predictors and responses. In many scientific problems, relationships are expressed through differential equations involving the variables, their derivatives, and other quantities. In more complex data-science problems, there may be no useful explicit analytical expression at all.

The relevant question is therefore:

How do we determine the model f?

There are several ways to approach this problem. Two useful perspectives are **theory-informed modelling** and **data-driven modelling**.

2.2 Theory-Informed Modelling

Suppose first that we have an established theory describing the system.

For the harmonic oscillator, classical mechanics provides such a theory. According to Newton's second law of motion, the net force acting on an object is related to its acceleration by

$$\mathbf{F} = m\mathbf{a}. \tag{2}$$

Newton's second law does not, by itself, specify the particular force acting on the mass. For that we need an additional description of the spring.

Within its elastic regime, Hooke's law states that the restoring force exerted by a spring is proportional to its displacement from equilibrium and acts in the opposite direction:

$$F = -kx. \tag{3}$$

Because the motion is one-dimensional and acceleration is the second derivative of position with respect to time,

$$a(t) = \frac{d^2x}{dt^2} = \ddot{x}(t). \tag{4}$$

Combining Equation 2 and Equation 3 gives the equation of motion

$$m\frac{d^2x}{dt^2} = -kx. \tag{5}$$

Defining the angular frequency ω by

$$\omega^2 := \frac{k}{m}, \tag{6}$$

the equation of motion becomes

$$\ddot{x} + \omega^2 x = 0. \tag{7}$$

Its general solution is

$$x(t) = A\cos(\omega t) + B\sin(\omega t), \tag{8}$$

where A and B are constants determined by the initial conditions.

Suppose that the object is initially displaced to its maximum position and released from rest:

$$x(0) = x_{\text{max}}, \qquad v(0) = 0, \tag{9}$$

where

$$v(t) = \frac{dx}{dt}. \tag{10}$$

Applying the first initial condition to Equation 8,

$$x(0) = A\cos(0) + B\sin(0) = A, \tag{11}$$

and therefore

$$A = x_{\text{max}}. \tag{12}$$

Differentiating Equation 8 gives

$$v(t) = -A\omega\sin(\omega t) + B\omega\cos(\omega t). \tag{13}$$

Evaluating this expression at $t = 0$,

$$v(0) = B\omega. \tag{14}$$

Since $v(0) = 0$ and $\omega \neq 0$,

$$B = 0. \tag{15}$$

The particular solution is therefore

$$x(t) = x_{\text{max}}\cos(\omega t). \tag{16}$$

We have obtained the functional relationship introduced in Equation 1:

$$f(t) = x_{\text{max}}\cos(\omega t). \tag{17}$$

The important point here is not the harmonic oscillator itself, but **how the model was obtained**.

We began with general theoretical principles — Newton's laws and Hooke's law — and proceeded deductively. The theory constrained the possible behaviour of the system, led to a differential equation, and ultimately determined a functional form for the relationship between position and time.

The resulting model can then be compared with experimental observations. For example, rather than assuming that the initial conditions are known exactly, we could fit the more general model

$$x(t) = A\cos(\omega t + \phi) \tag{18}$$

to the observations in Table 1 using a suitable estimation procedure, such as least squares. This would provide estimates $\widehat{A}$, $\widehat{\omega}$, and $\widehat{\phi}$.

The estimated angular frequency could then be compared with the theoretical value

$$\omega = \sqrt{\frac{k}{m}}, \tag{19}$$

where k and m are obtained from independent measurements.

This illustrates an important distinction:

> **Model specification and model fitting are not the same thing.**

In this example, physical theory largely determines the **structure of the model**. The experimental data then allow us to **estimate its unknown parameters** and assess whether its predictions are consistent with the observed behaviour of the system.

The same equation of motion could also be obtained through a different theoretical framework, such as Lagrangian mechanics. The derivation would be different, but the basic modelling strategy would remain the same: prior theoretical knowledge places strong constraints on the class of models that we consider plausible.

At a high level, this approach can be summarised as

$$\begin{array}{c} \text{Theory} \longrightarrow \text{General principles} \longrightarrow \text{Mathematical model} \\ \Downarrow \\ \text{Functional relationship} \longrightarrow \text{Fit to data} \longrightarrow \text{Empirical evaluation} \end{array}$$

2.3 Data-Driven Modelling

Now consider the same experiment from a different perspective.

Suppose that we know nothing about Newtonian mechanics, Hooke's law, or the differential equation governing the system. We are given only the observations in Table 1.

What can we learn from the data alone?

A natural first step is to visualise the observations by plotting position against time. In a real experiment, measurements would contain some degree of noise, so we might obtain a pattern similar to that shown in Figure 4.

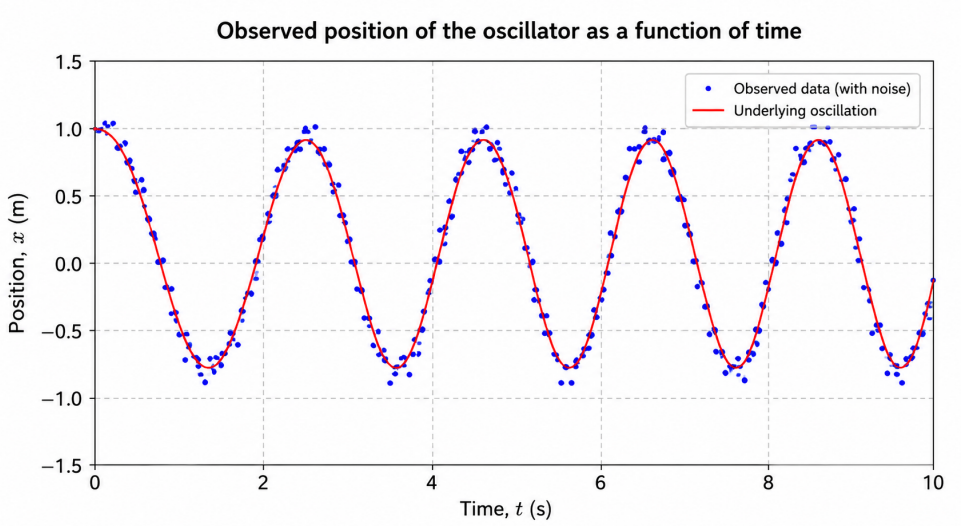

Figure 4: Observed position of the oscillator as a function of time, including measurement noise.

Examining the plot reveals several patterns. The position oscillates between approximately constant upper and lower values, the period appears approximately constant, and the object is close to its maximum displacement at $t = 0$.

There is clearly structure in the data even though we do not know the physical mechanism responsible for it.

At this point, we can follow different data-driven modelling strategies.

2.3.1 Prespecifying a Functional Form

One possibility is to use the observed patterns, together with our mathematical knowledge, to propose a plausible functional relationship.

The periodic behaviour might suggest a trigonometric function such as

$$f(t; A, \omega, \phi) = A\cos(\omega t + \phi). \tag{20}$$

This looks very similar to the model obtained previously, but there is an important difference.

In the theory-informed approach, the trigonometric form emerged **deductively** from the differential equation governing the physical system. Here, we have proposed it **inductively** from patterns observed in the data.

Once the functional form has been specified, we can estimate its unknown parameters from the observations:

$$\hat{\theta} = (\hat{A}, \hat{\omega}, \hat{\phi}). \tag{21}$$

For example, we could estimate them by minimising the residual sum of squares:

$$\hat{\theta} = \arg\min_{\theta} \sum_{i=1}^{N} \left[x_i - f(t_i; \theta)\right]^2. \tag{22}$$

We can then evaluate how well the fitted model describes observations that were not used to estimate its parameters.

Although this is a data-driven approach, it still incorporates substantial prior knowledge. We have explicitly chosen the mathematical form of the relationship. The data determine the parameter estimates, but the analyst has determined the basic structure of the model.

2.3.2 Learning a Flexible Relationship from Data

A different strategy is to avoid specifying an explicit functional relationship such as a cosine in advance.

Instead, we select a sufficiently flexible **model class** and use the observed data to learn a particular function within that class.

We can represent this abstractly as

$$\hat{x} = f_{\theta}(t), \tag{23}$$

where θ represents the quantities learned during training.

A neural network is one example. Neural networks are **parametric models** — often with very large numbers of parameters — but we do not generally need to prescribe an explicit analytical relationship between predictor and response such as

$$x(t) = A\cos(\omega t + \phi).$$

Instead, we specify an architecture and a learning procedure, and the parameters are estimated from the observations. With a squared-error objective, the training problem could be written abstractly as

$$\widehat{\theta} = \arg\min_{\theta} \sum_{i=1}^{N} \left[x_i - f_\theta(t_i)\right]^2. \tag{24}$$

Decision trees, random forests, boosting methods, neural networks, splines, and many other statistical and machine-learning methods provide different ways of learning flexible relationships from data without requiring the analyst to specify a simple explicit equation relating predictors to responses.

This does not mean that these methods learn without assumptions.

Every modelling method restricts, in some way, the relationships that can be learned. A linear model assumes a particular linear structure. A decision tree represents relationships through recursive partitions of the predictor space. A neural network is constrained by its architecture, connectivity, activation functions, optimisation procedure, and other design choices.

These choices introduce what is commonly called **inductive bias**: the assumptions that make some solutions more likely or accessible to a learning algorithm than others.

Machine learning therefore does not remove assumptions from modelling. Instead, many machine-learning methods allow us to replace a strongly prespecified functional relationship with a more flexible class of possible relationships and let the data play a larger role in determining the particular function that is learned.

This data-driven approach can be summarised broadly as

$$\begin{array}{c} \text{Data} \longrightarrow \text{Observed patterns} \longrightarrow \text{Model class} \\ \Downarrow \\ \text{Learning from data} \longrightarrow \text{Empirical evaluation} \end{array}$$

2.4 Theory-Informed and Data-Driven Modelling

We can now summarise the two perspectives.

In **theory-informed modelling**, established knowledge about the underlying mechanism strongly constrains the mathematical structure of the model:

$$\text{Theory} \longrightarrow \text{General principles} \longrightarrow \text{Mathematical model}$$

$$\Downarrow$$

$$\text{Functional relationship} \longrightarrow \text{Parameter estimation} \longrightarrow \text{Empirical evaluation}$$

In **data-driven modelling**, the relationship is inferred primarily from observations:

$$\text{Data} \longrightarrow \text{Observed patterns}$$

$$\Downarrow$$

$$\begin{cases} \text{Prespecified functional form} \longrightarrow \text{Parameter estimation} \\ \text{Flexible model class} \longrightarrow \text{Learning from data} \end{cases}$$

$$\Downarrow$$

$$\text{Empirical evaluation}$$

These approaches are not mutually exclusive. They are better understood as different ways of incorporating available information into the modelling process.

In practice, many models lie somewhere between the two. Domain knowledge may guide feature selection or feature engineering, constrain parameter values, inform the choice of model class, or rule out relationships that are scientifically or operationally implausible.

The key distinction is therefore not whether assumptions are present — **all models make assumptions** — but how much structure is specified before learning from the data and where that structure comes from.

This distinction becomes particularly relevant when we move from physical systems to many real-world data-science problems.

In the harmonic oscillator, we have well-established physical laws that provide a strong theoretical description of the underlying mechanism. In many business problems, no comparable theory exists.

Consider **credit-card fraud detection**. There is no equivalent of Newton's laws from which we can derive an equation governing whether a transaction will be fraudulent. Similarly, in **customer churn prediction**, there is no fundamental theory from which individual customer behaviour can be deduced.

Instead, we have data: transaction histories, customer characteristics, behavioural variables, temporal information, previous outcomes, and many other potentially useful observations.

The modelling problem is therefore different. We want to use the information contained in those observations to learn relationships that remain useful when applied to cases that were not part of the original dataset.

This is where **machine learning** becomes particularly useful.

Machine-learning methods provide systematic ways of learning relationships from data when the underlying mechanism is unknown, too complex to derive analytically, or not conveniently represented by a prespecified functional form.

However, this flexibility introduces an important difficulty. A sufficiently flexible model may learn not only meaningful structure but also noise, accidental regularities, or peculiarities of the particular dataset used to train it.

The central question is therefore not simply

> **Can the model fit the observed data?**

but rather

> **Can the model learn a relationship from observed data that remains useful on data it has never seen?**

This ability is known as **generalisation**, and it is one of the central problems in machine learning.

It provides the starting point for understanding how machine-learning models are trained, evaluated, selected, and ultimately used in practice.

3 Machine Learning Modelling

The previous section introduced data-driven modelling as a way of learning relationships from observations when an explicit theoretical description of the underlying mechanism is unavailable, incomplete, or impractical. We can now formulate this problem more precisely.

Suppose that we observe N instances, each described by M predictors. For the i-th observation, we collect the predictor values into the vector

$$\mathbf{x}_i = \begin{pmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{iM} \end{pmatrix} = (x_{i1}, x_{i2}, \dots, x_{iM})^\top \in \mathbb{R}^M. \tag{25}$$

The complete set of predictors can then be arranged into the matrix

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_N^\top \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1M} \\ x_{21} & x_{22} & \cdots & x_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{NM} \end{pmatrix} \in \mathbb{R}^{N \times M}. \tag{26}$$

Each row of $\mathbf{X}$ represents one observation, while each column represents one predictor. In other words, $\mathbf{x}_i^\top$ is the i-th row of the predictor matrix.

If a response is available for each observation, we denote the individual response by y_i and collect all responses into the vector

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N. \tag{27}$$

The nature of y_i depends on the problem. It may represent a numerical quantity, such as house price, energy consumption, demand, or the position of the harmonic oscillator considered previously. Alternatively, it may indicate membership in one of a finite set of classes, such as whether a transaction is legitimate or fraudulent. These two cases lead naturally to **regression** and **classification**, respectively, which will be discussed in the next section.

Before considering these learning problems separately, it is useful to formulate the modelling problem in general terms.

3.1 Learning an Unknown Relationship

Consider first a regression problem. We assume that the response can be represented as

$$y_i = f(\mathbf{x}_i) + \varepsilon_i, \tag{28}$$

where f is a fixed but unknown function relating the predictors to the response, while ε_i represents the component of the response that is not explained by the predictors available to the model.

A common assumption is that the error has zero conditional mean,

$$\mathbb{E}\left[\varepsilon_i \mid \mathbf{x}_i\right] = 0. \tag{29}$$

In the simplest homoscedastic setting, we additionally assume that

$$\text{Var}\left(\varepsilon_i \mid \mathbf{x}_i\right) = \sigma^2. \tag{30}$$

Under the zero conditional mean assumption, $f(\mathbf{x}_i)$ represents the conditional expected value of the response. Indeed,

$$\begin{aligned} \mathbb{E}\left[y_i \mid \mathbf{x}_i\right] &= \mathbb{E}\left[f(\mathbf{x}_i) + \varepsilon_i \mid \mathbf{x}_i\right] \\ &= f(\mathbf{x}_i) + \mathbb{E}\left[\varepsilon_i \mid \mathbf{x}_i\right] \\ &= f(\mathbf{x}_i). \end{aligned} \tag{31}$$

The function f therefore represents the systematic relationship between the available predictors and the response. In practice, however, f is unknown. Given a training dataset

$$\mathcal{D} = \left\{(\mathbf{x}_i, y_i)\right\}_{i=1}^{N}, \tag{32}$$

our objective is to construct an estimate $\hat{f}$ of the unknown function f. For a previously unseen observation $\mathbf{x}_0$, the corresponding prediction is

$$\hat{y}_0 = \hat{f}(\mathbf{x}_0). \tag{33}$$

This provides a useful working description of supervised machine learning: given observed predictor-response pairs, we attempt to learn a relationship from the training data that remains useful when applied to previously unseen observations.

The last part of this statement is fundamental. The objective is not merely to reproduce the training observations. A model that fits its training data extremely well but performs poorly when presented with new observations has limited predictive value. The central objective is therefore **generalisation**.

3.2 The Nature of the Data

The notation $\mathbf{x}_i \in \mathbb{R}^M$ may suggest that machine-learning datasets always begin as conventional tables containing numerical columns. This is not the case. The mathematical representation used by a learning algorithm depends on the nature of the information being analysed.

In a conventional business problem, an observation may naturally be represented as a collection of variables. A customer, for example, may be described by demographic, transactional, and behavioural information, while a financial transaction may contain an amount, timestamp, merchant information, location, and characteristics derived from previous transactions.

Other types of data require different representations. Images consist of numerical pixel values arranged spatially and form the natural input to **Computer Vision** problems. Text consists of sequences of linguistic units that can be transformed into numerical representations for **Natural Language Processing (NLP)**. Modern **Large Language Models (LLMs)** provide a particularly prominent example of learning from very large collections of textual data.

The purpose of mentioning these examples here is not to develop those fields separately, but to emphasise a general point: **the nature and structure of the observed data strongly influence how the learning problem should be represented and which modelling approaches are appropriate**.

The abstract notation $\mathbf{x}_i$ should therefore be understood as a mathematical representation of the information available to the learning algorithm, rather than necessarily as a row from an ordinary spreadsheet.

3.3 Reducible and Irreducible Prediction Error

The model in Equation 28 also reveals an important limitation of prediction.

Consider a particular unseen predictor vector $\mathbf{x}_0$, whose response is

$$y_0 = f(\mathbf{x}_0) + \varepsilon_0. \tag{34}$$

Suppose for the moment that $\mathbf{x}_0$ and the fitted function $\hat{f}$ are fixed, so that the only remaining source of variability is ε_0. The squared prediction error is $\left[y_0 - \hat{f}(\mathbf{x}_0)\right]^2$. Taking its expected value with respect to ε_0 gives

$$\mathbb{E}_{\varepsilon_0}\left[\left(y_0 - \hat{f}(\mathbf{x}_0)\right)^2\right] = \mathbb{E}_{\varepsilon_0}\left[\left(f(\mathbf{x}_0) + \varepsilon_0 - \hat{f}(\mathbf{x}_0)\right)^2\right].$$

Expanding the square,

$$\begin{aligned}\mathbb{E}_{\varepsilon_0}\left[\left(y_0 - \hat{f}(\mathbf{x}_0)\right)^2\right] = \mathbb{E}_{\varepsilon_0}&\left[\left(f(\mathbf{x}_0) - \hat{f}(\mathbf{x}_0)\right)^2\right] \\ &+ 2\mathbb{E}_{\varepsilon_0}\left[\left(f(\mathbf{x}_0) - \hat{f}(\mathbf{x}_0)\right)\varepsilon_0\right] \\ &+ \mathbb{E}_{\varepsilon_0}\left[\varepsilon_0^2\right].\end{aligned} \tag{35}$$

Since $f(\mathbf{x}_0)$ and $\hat{f}(\mathbf{x}_0)$ are fixed with respect to ε_0,

$$\mathbb{E}_{\varepsilon_0}\left[\left(f(\mathbf{x}_0) - \hat{f}(\mathbf{x}_0)\right)^2\right] = \left(f(\mathbf{x}_0) - \hat{f}(\mathbf{x}_0)\right)^2.$$

For the cross term,

$$\begin{aligned}\mathbb{E}_{\varepsilon_0}\left[\left(f(\mathbf{x}_0)-\hat{f}(\mathbf{x}_0)\right)\varepsilon_0\right] &= \left(f(\mathbf{x}_0)-\hat{f}(\mathbf{x}_0)\right)\mathbb{E}_{\varepsilon_0}[\varepsilon_0] \\ &= 0.\end{aligned}$$

Finally, using the identity $\mathrm{Var}(Z) = \mathbb{E}[Z^2] - \mathbb{E}[Z]^2$,

$$\begin{aligned}\mathbb{E}\left[\varepsilon_0^2\right] &= \mathrm{Var}(\varepsilon_0) + \left(\mathbb{E}[\varepsilon_0]\right)^2 \\ &= \sigma^2.\end{aligned}$$

Substituting these results into Equation 35,

$$\mathbb{E}_{\varepsilon_0}\left[\left(y_0-\hat{f}(\mathbf{x}_0)\right)^2\right] = \left[f(\mathbf{x}_0)-\hat{f}(\mathbf{x}_0)\right]^2 + \sigma^2. \tag{36}$$

The first term, $\left[f(\mathbf{x}_0)-\hat{f}(\mathbf{x}_0)\right]^2$, measures the discrepancy between the underlying relationship and our estimate of that relationship. This component is **reducible**, at least in principle. A more appropriate model, better data, more informative predictors, or an improved learning procedure may reduce it.

The second term, σ^2, is the **irreducible error** associated with the variability that remains after conditioning on the predictors available to the model.

Thus, the expected prediction error at $\mathbf{x}_0$ can be understood conceptually as

$$\text{Prediction Error} = \text{Reducible Error} + \text{Irreducible Error}. \tag{37}$$

The term *irreducible* must be interpreted relative to the information available to the model. Some variation assigned to ε may arise because relevant variables were not measured or were omitted from the predictor set. If additional informative predictors become available, part of what was previously treated as unexplained variability may become predictable.

Other components may arise from measurement error or from variability that is genuinely unpredictable at the level at which the problem is being modelled.

The practical objective is therefore not necessarily to drive prediction error to zero. Rather, it is to reduce the component that can reasonably be reduced while recognising the uncertainty that remains.

3.4 Learning a Model from Data

Once a model class has been selected, learning generally consists of determining the particular model within that class that best satisfies a specified objective.

For a parametrised model, we write $f_\theta(\mathbf{x})$, where

$$\theta = (\theta_0, \theta_1, \dots, \theta_P)^\top \tag{38}$$

denotes the parameters to be learned from the training data.

Training can often be expressed abstractly as an optimisation problem:

$$\hat{\theta} = \arg\min_\theta \mathcal{L}\left(\theta; \mathbf{X}, \mathbf{y}\right), \tag{39}$$

where $\mathcal{L}$ is an objective function that quantifies how well a particular choice of θ satisfies the learning objective.

Once $\hat{\theta}$ has been estimated, the fitted model is $\hat{f}(\mathbf{x}) = f_{\hat{\theta}}(\mathbf{x})$.

The precise objective function depends on the modelling problem. For ordinary linear regression, for example, we may write

$$f_\theta(\mathbf{x}_i) = \theta_0 + \theta_1 x_{i1} + \cdots + \theta_M x_{iM}, \tag{40}$$

and estimate the parameters by minimising the **Residual Sum of Squares (RSS)**:

$$\mathrm{RSS}(\theta) = \sum_{i=1}^{N} \left[y_i - f_\theta(\mathbf{x}_i)\right]^2. \tag{41}$$

The parameter estimate is therefore

$$\hat{\theta} = \arg\min_\theta \mathrm{RSS}(\theta). \tag{42}$$

Other methods use different objective functions. Logistic regression, for example, is commonly fitted by maximum likelihood, or equivalently by minimising the negative log-likelihood. Neural networks may use mean squared error for regression problems or cross-entropy for classification problems, among many other possible objectives.

Although the details differ, the general learning process can be represented schematically as

$$\text{Choose a model class} \longrightarrow \text{Define a learning objective}$$

$$\Downarrow$$

$$\text{Optimise the objective} \longrightarrow \text{Obtain a fitted model} \longrightarrow \text{Evaluate generalisation.}$$

This perspective also helps explain why machine learning draws naturally from several technical disciplines. **Statistics** provides tools for estimation, uncertainty, inference, and the analysis of data; **numerical analysis and optimisation** provide methods for solving estimation problems that may not admit convenient analytical solutions; and **computer science** provides the algorithms, data structures, and computational systems required to implement these methods efficiently and at scale.

These disciplines overlap substantially, and machine learning should not be understood simply as their arithmetic sum. Rather, modern machine learning draws on each of them to learn useful relationships from data and, crucially, to determine whether those relationships remain useful beyond the observations on which they were learned.

4 Classification of Machine Learning Algorithms

Machine-learning methods can be classified in different ways depending on the nature of the data, the information available during training, and the objective of the learning process.

At a broad level, three major learning paradigms are commonly distinguished:

- **Supervised Learning (SL)**
- **Unsupervised Learning (UL)**
- **Reinforcement Learning (RL)**

The distinction between them is not primarily determined by the particular algorithm being used, but by the structure of the learning problem.

In supervised learning, the model learns from observations for which a response is available. In unsupervised learning, no response is provided and the objective is to identify structure in the observed data. Reinforcement learning considers a different setting in which an agent learns through interaction with an environment and the consequences of its actions.

These three paradigms provide a useful general framework for organising machine-learning problems.

4.1 Supervised Learning

In **supervised learning**, the training dataset consists of predictor-response pairs:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}. \tag{43}$$

The predictor vector $\mathbf{x}_i$ contains the information available for the i-th observation, while y_i provides the corresponding response that the model is expected to learn to predict.

The objective is to use the training data to estimate a relationship between predictors and response that can subsequently be applied to previously unseen observations.

As introduced in the previous section, this relationship can be represented abstractly as

$$y_i = f(\mathbf{x}_i) + \varepsilon_i,$$

and the learning procedure produces an estimate $\hat{f}$ of the unknown relationship f.

For a new observation $\mathbf{x}_0$, the model produces the prediction $\hat{y}_0 = \hat{f}(\mathbf{x}_0)$.

The nature of the response y leads to two fundamental types of supervised-learning problems: **regression** and **classification**.

4.1.1 Regression

In a **regression problem**, the response represents a numerical quantity that can vary over a range of values.

Examples include predicting house prices, estimating energy consumption, forecasting demand, predicting temperature, or estimating the position of the harmonic oscillator considered previously.

The objective is to learn a function

$$\hat{f} : \mathcal{X} \longrightarrow \mathbb{R} \tag{44}$$

that maps the predictor information to a numerical prediction.

The simplest and most fundamental regression model is **linear regression**. For M predictors, it assumes a relationship of the form

$$y_i = \theta_0 + \theta_1 x_{i1} + \theta_2 x_{i2} + \cdots + \theta_M x_{iM} + \varepsilon_i. \tag{45}$$

Using the parameter vector

$$\theta = (\theta_0, \theta_1, \dots, \theta_M)^\top,$$

the model is completely determined once its parameters have been estimated from the training data.

Linear regression is particularly useful as a starting point because it is mathematically transparent, computationally inexpensive, and relatively easy to interpret. It also provides the foundation for understanding many concepts that appear throughout statistical learning, including parameter estimation, residuals, loss functions, model assumptions, inference, and regularisation.

However, the relationship between predictors and response need not be linear. More flexible regression methods include regression trees, random forests, boosting methods, support vector regression, spline-based models, and neural networks.

Despite their differences, these methods address the same general problem: estimating a numerical response from the information contained in the predictors.

4.1.2 Classification

In a **classification problem**, the response indicates membership in one of a finite set of classes.

For a binary classification problem, for example,

$$y_i \in \{0, 1\}. \tag{46}$$

In credit-card fraud detection, we might define

$$y_i = \begin{cases} 0, & \text{legitimate transaction,} \\ 1, & \text{fraudulent transaction.} \end{cases} \tag{47}$$

The learning problem is therefore different from regression. Rather than estimating an unrestricted numerical response, the objective is to determine which class is most appropriate for a new observation.

Conceptually, a classifier defines a mapping

$$\hat{f} : \mathcal{X} \longrightarrow \mathcal{C}, \tag{48}$$

where $\mathcal{C}$ denotes the set of possible classes.

For binary classification,

$$\mathcal{C} = \{0, 1\}.$$

Many classification algorithms approach the problem by estimating class probabilities rather than assigning a class directly. In the binary case, for example, a model may estimate

$$P\left(Y = 1 \mid \mathbf{X} = \mathbf{x}\right). \tag{49}$$

A classification rule can then be constructed from the estimated probability. For example, using a threshold c,

$$\hat{y} = \begin{cases} 1, & \hat{P}(Y = 1 \mid \mathbf{X} = \mathbf{x}) \geq c, \\ 0, & \hat{P}(Y = 1 \mid \mathbf{X} = \mathbf{x}) < c. \end{cases} \tag{50}$$

The threshold need not necessarily be 0.5. Its appropriate value depends on the problem, the relative consequences of different types of error, and the purpose for which the model will be used.

A fundamental baseline for binary classification is **logistic regression**. Despite its name, logistic regression is used primarily as a classification method because it models the probability that an observation belongs to a particular class.

The model can be written as

$$P\left(Y = 1 \mid \mathbf{X} = \mathbf{x}\right) = \frac{1}{1 + \exp\left[-\left(\theta_0 + \sum_{j=1}^{M} \theta_j x_j\right)\right]}. \tag{51}$$

Equivalently, if $\tilde{\mathbf{x}} = (1, x_1, \ldots, x_M)^\top$ includes the intercept term,

$$P\left(Y = 1 \mid \mathbf{X} = \mathbf{x}\right) = \frac{1}{1 + \exp(-\theta^\top \tilde{\mathbf{x}})}. \tag{52}$$

The logistic function maps any real-valued linear combination of the predictors into the interval $(0, 1)$, allowing the result to be interpreted as a probability.

Other classification methods include decision trees, random forests, boosting methods, support vector machines, nearest-neighbour methods, and neural networks.

Regression and classification can therefore be summarised schematically as

$$\text{Supervised Learning}$$

$$\Downarrow$$

$$\begin{cases} \text{Regression} & \longrightarrow \text{ Numerical response} \\ \text{Classification} & \longrightarrow \text{ Class membership} \end{cases}$$

The distinction is conceptually simple but important: **regression and classification differ primarily in the nature of the response and consequently in the learning objective, not necessarily in the complexity of the algorithm**.

Indeed, several families of models can be adapted to both problems. Trees, random forests, boosting methods, support vector machines, and neural networks all have regression and classification variants.

4.2 Unsupervised Learning

In **unsupervised learning**, no response variable is supplied with the observations. The dataset therefore has the form

$$\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^{N} . \tag{53}$$

There is no y_i whose relationship with the predictors can be learned. The objective is instead to identify useful structure, regularities, or representations within the observed data themselves.

This makes unsupervised learning fundamentally different from supervised prediction. There is no externally supplied response against which a prediction can be directly compared.

Two important unsupervised-learning problems are **clustering** and **dimensionality reduction**.

4.2.1 Clustering

Clustering attempts to organise observations into groups such that observations within the same group are, according to some criterion, more similar to one another than to observations belonging to other groups.

Starting from

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} ,$$

the objective is to identify a collection of groups

$$C_1, C_2, \ldots, C_K$$

from the structure of the data themselves.

Unlike classification, these groups are not supplied beforehand as known responses. They are inferred from patterns present in the predictor space.

This distinction is important. If observations have already been labelled as belonging to classes and the objective is to predict those labels, the problem is classification. If no such labels exist and the objective is to discover groups from the observations themselves, the problem is clustering.

Two classical examples are **K-means clustering** and **hierarchical clustering**.

Clustering can be useful for exploratory analysis, customer segmentation, grouping documents or products, identifying patterns in scientific data, and discovering potentially meaningful subpopulations before a more specific modelling problem has been formulated.

4.2.2 Dimensionality Reduction

Datasets with many predictors can contain substantial redundancy. Several variables may describe similar aspects of the observations, or much of the variation in the data may lie approximately within a lower-dimensional structure.

Dimensionality reduction attempts to construct a representation

$$\mathbf{x}_i \in \mathbb{R}^M \quad \longrightarrow \quad \mathbf{z}_i \in \mathbb{R}^K, \qquad K < M, \tag{54}$$

while preserving information or structure considered relevant to the problem.

A classical example is **Principal Component Analysis (PCA)**, which constructs new orthogonal directions that successively capture as much variance in the data as possible.

Dimensionality reduction may be useful for visualisation, data compression, noise reduction, exploratory analysis, feature construction, or as a preprocessing stage for another learning algorithm.

The key distinction from supervised learning remains the same: the transformation is learned from the structure of the predictors themselves rather than from their relationship with an observed response.

4.3 Reinforcement Learning

Reinforcement Learning (**RL**) considers a substantially different learning problem.

Rather than learning from a fixed collection of predictor-response pairs, an **agent** interacts sequentially with an **environment**.

At time t, the agent observes information about the current state s_t and selects an action a_t. As a consequence of that action, the environment produces a reward r_{t+1} and transitions to a new state s_{t+1}.

The interaction can be represented schematically as

$$(s_t, a_t) \longrightarrow (r_{t+1}, s_{t+1}). \tag{55}$$

The objective is to learn how actions should be selected in different states in order to maximise reward over time.

The agent's behaviour is described by a **policy** π, which can be represented probabilistically as

$$\pi(a \mid s) = P(A_t = a \mid S_t = s). \tag{56}$$

Unlike supervised learning, the agent is not normally given the correct action for each state. Instead, it must learn from the consequences of the actions it takes.

This introduces an important additional difficulty: the actions chosen by the learner influence the future observations and rewards that become available.

A simplified representation of the learning process is

$$\begin{array}{c} \text{State} \longrightarrow \text{Agent} \longrightarrow \text{Action} \\ \Downarrow \\ \text{Environment} \longrightarrow \text{Reward and New State} \longrightarrow \text{Agent}. \end{array}$$

Reinforcement learning is particularly relevant to sequential decision-making problems such as robotics, control systems, game-playing agents, resource allocation, and other situations in which present decisions affect future outcomes.

For the purposes of these notes, the important point is the conceptual distinction among the three broad learning paradigms:

- **Supervised Learning:** learn a relationship using observed predictors and responses.

- **Unsupervised Learning:** discover structure from observations without an associated response.
- **Reinforcement Learning:** learn actions through sequential interaction with an environment and the rewards generated by those actions.

4.4 Parametric and Non-Parametric Models

The distinction between supervised, unsupervised, and reinforcement learning describes the **learning setting**. A different classification concerns the mathematical structure of the models used within those settings.

One traditional distinction is between **parametric** and **non-parametric** methods.

4.4.1 Parametric Models

A **parametric model** represents the relationship to be learned through a fixed finite-dimensional parameter vector

$$\theta = (\theta_0, \theta_1, \dots, \theta_P)^\top.$$

Once the model structure has been specified, the number of parameters is fixed and learning consists primarily of estimating their values from the available data.

Linear regression provides the simplest example:

$$f_\theta(\mathbf{x}) = \theta_0 + \sum_{j=1}^{M} \theta_j x_j. \tag{57}$$

Once $\hat{\theta}$ has been estimated, the fitted relationship is completely specified.

Parametric models need not be simple. A neural network is also a parametric model. For a fixed architecture, it contains a finite number of parameters, even though that number may be extremely large.

This is an important distinction:

> **Parametric does not mean simple, and it does not mean that the model contains only a small number of parameters.**

What makes the model parametric is that, once its structure is fixed, it is represented by a finite-dimensional parameter vector.

4.4.2 Non-Parametric Models

The term **non-parametric** can be misleading because it does not mean that the method contains no parameters, hyperparameters, or quantities that must be determined.

Rather, a non-parametric method does not restrict the relationship being learned to a fixed finite-dimensional parameterisation whose dimension remains unchanged as the amount of available data increases.

Its effective complexity can therefore grow with the training data.

Nearest-neighbour methods provide an intuitive example. In **K-Nearest Neighbours** (**KNN**), prediction for a new observation depends directly on observations retained from the training set rather than on reducing the entire fitted relationship to a fixed vector $\hat{\theta}$.

Other examples include several kernel-based and spline-based methods.

Non-parametric methods are often more flexible because they impose fewer restrictions on the form of the relationship that can be learned. This flexibility, however, generally comes with requirements of its own, including sufficient data and appropriate control of model complexity.

4.5 Functional Form and Model Flexibility

The parametric/non-parametric distinction should not be confused with the earlier distinction between specifying an explicit functional form and using a flexible model class.

For example, suppose we explicitly assume

$$f(t) = \theta_0 \cos(\theta_1 t).$$

This is a parametric model with a clearly prescribed analytical form.

A deep neural network is also parametric, but its functional form is not usually specified at the level of an explicit relationship between the original predictors and response such as the expression above. Instead, we specify an architecture containing many elementary transformations and learn a potentially very complex function through the parameters of that architecture.

Thus, both models are parametric, but the nature of the assumptions imposed on the relationship is very different.

This is why, in practical machine learning, the distinction between parametric and non-parametric methods is useful but does not by itself describe how restrictive or flexible a model is.

A more general question is:

> **What family of relationships can the model represent, and what assumptions constrain what it can learn?**

Every learning method imposes some assumptions or preferences on the possible relationships that can be learned. These constitute its **inductive bias**.

Without such restrictions, learning from finite data would not be possible in any meaningful sense: infinitely many relationships could be made consistent with a finite collection of observations.

Model assumptions are therefore not inherently undesirable. The practical issue is whether those assumptions are appropriate for the problem and whether the resulting model generalises to observations beyond the training sample.

This brings us to one of the central problems in machine learning. Increasing model flexibility can improve our ability to capture genuine structure in the data, but it can also increase sensitivity to the particular observations used during training.

The consequences of this tension are **underfitting**, **overfitting**, and the **bias–variance trade-off**, which provide the starting point for understanding the scope and limitations of machine-learning models.

5 Scope and Limitations of ML Models

The flexibility of machine-learning models is one of their main strengths. It allows them to represent relationships that may be difficult to derive analytically or describe through a simple prespecified functional form.

That flexibility, however, also introduces an important difficulty.

A model may fit the available training data extremely well and still perform poorly when applied to new observations. Training performance alone is therefore not sufficient to assess whether a useful relationship has been learned.

The central question is one of **generalisation**:

> **Can the model learn a relationship from the observed data that remains useful when applied to observations it has not seen before?**

Two common failure modes illustrate this problem: **underfitting** and **overfitting**.

5.1 Underfitting and Overfitting

A model **underfits** when it is too restrictive to capture important structure in the data.

Suppose, for example, that the underlying relationship between a predictor and a response is strongly nonlinear, but we insist on fitting a linear relationship. Even if the model parameters are estimated optimally, the model class itself may be incapable of representing the relevant structure.

The resulting model will generally perform poorly on both the training data and unseen observations.

Underfitting is therefore associated with insufficient model flexibility.

At the opposite extreme, a model **overfits** when it adapts too closely to the particular training dataset.

A sufficiently flexible model may capture not only the systematic relationship between predictors and response but also noise, accidental fluctuations, unusual observations, or patterns that are specific to the training sample.

In that case, the training error may become very small while prediction performance on unseen observations deteriorates.

The distinction can therefore be summarised as follows:

- **Underfitting:** the model fails to capture relevant structure in the data.
- **Overfitting:** the model captures structure that is specific to the training sample and does not generalise.

The objective is not to choose the model that fits the training data most closely. Rather, a useful model must be flexible enough to learn meaningful structure while remaining sufficiently constrained to generalise beyond the observations used for fitting.

This tension can be studied more formally through the **bias–variance trade-off**.

5.2 Bias–Variance Trade-off

The fitted model obtained from a learning algorithm depends on the particular training dataset used.

Suppose that the same underlying data-generating process could be observed repeatedly, producing different training datasets

$$\mathcal{D}_1, \mathcal{D}_2, \ldots$$

drawn from the same population.

If the same learning procedure were fitted independently to each of these datasets, we would generally obtain different fitted functions,

$$\hat{f}_{\mathcal{D}_1}, \hat{f}_{\mathcal{D}_2}, \ldots$$

because the observations contained in each training sample would differ.

This variability in the fitted model is central to understanding generalisation error.

Consider a fixed unseen predictor vector $\mathbf{x}_0$. Suppose that its response is generated according to

$$y_0 = f(\mathbf{x}_0) + \varepsilon_0, \tag{58}$$

where

$$\mathbb{E}[\varepsilon_0] = 0$$

and

$$\text{Var}(\varepsilon_0) = \sigma^2.$$

For a model fitted on a random training dataset $\mathcal{D}$, the prediction at $\mathbf{x}_0$ is $\hat{f}_{\mathcal{D}}(\mathbf{x}_0)$.

We are interested in the expected squared prediction error at $\mathbf{x}_0$:

$$\text{Err}(\mathbf{x}_0) = \mathbb{E}_{\mathcal{D},\varepsilon_0}\left[\left(y_0 - \hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right)^2\right]. \tag{59}$$

Substituting Equation 58,

$$\text{Err}(\mathbf{x}_0) = \mathbb{E}_{\mathcal{D},\varepsilon_0}\left[\left(f(\mathbf{x}_0) + \varepsilon_0 - \hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right)^2\right]. \tag{60}$$

Expanding the square,

$$\begin{aligned}\mathrm{Err}(\mathbf{x}_0) = \mathbb{E}\left[\left(f(\mathbf{x}_0) - \hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right)^2\right] \\ + 2\mathbb{E}\left[\left(f(\mathbf{x}_0) - \hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right)\varepsilon_0\right] \\ + \mathbb{E}\left[\varepsilon_0^2\right].\end{aligned} \tag{61}$$

Because the noise associated with the unseen observation has zero mean and is independent of the fitted model,

$$\begin{aligned}\mathbb{E}\left[\left(f(\mathbf{x}_0) - \hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right)\varepsilon_0\right] &= \mathbb{E}_{\mathcal{D}}\left[f(\mathbf{x}_0) - \hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right]\mathbb{E}_{\varepsilon_0}[\varepsilon_0] \\ &= 0.\end{aligned}$$

Also,

$$\begin{aligned}\mathbb{E}\left[\varepsilon_0^2\right] &= \mathrm{Var}(\varepsilon_0) + \left(\mathbb{E}[\varepsilon_0]\right)^2 \\ &= \sigma^2.\end{aligned}$$

Therefore,

$$\mathrm{Err}(\mathbf{x}_0) = \mathbb{E}_{\mathcal{D}}\left[\left(f(\mathbf{x}_0) - \hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right)^2\right] + \sigma^2. \tag{62}$$

The remaining expectation describes how much the fitted model differs from the underlying relationship because the training sample varies.

To decompose this term, define for simplicity

$$f_0 := f(\mathbf{x}_0)$$

and

$$\hat{f}_0 := \hat{f}_{\mathcal{D}}(\mathbf{x}_0).$$

Then we need to analyse

$$\mathbb{E}_{\mathcal{D}}\left[(f_0 - \hat{f}_0)^2\right].$$

Add and subtract $\mathbb{E}_{\mathcal{D}}[\hat{f}_0]$ inside the difference:

$$f_0 - \hat{f}_0 = f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0] + \mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0.$$

Squaring both sides,

$$\begin{aligned}(f_0 - \hat{f}_0)^2 = &\left(f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)^2 \\ &+ 2\left(f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0\right) \\ &+ \left(\mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0\right)^2.\end{aligned} \tag{63}$$

Now take expectation with respect to the training dataset:

$$\begin{aligned}\mathbb{E}_{\mathcal{D}}\left[(f_0 - \hat{f}_0)^2\right] = &\left(f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)^2 \\ &+ 2\left(f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)\mathbb{E}_{\mathcal{D}}\left[\mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0\right] \\ &+ \mathbb{E}_{\mathcal{D}}\left[\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0\right)^2\right].\end{aligned}$$

The middle term is zero because

$$\begin{aligned}\mathbb{E}_{\mathcal{D}}\left[\mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0\right] &= \mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \mathbb{E}_{\mathcal{D}}[\hat{f}_0] \\ &= 0.\end{aligned}$$

Hence,

$$\begin{aligned}\mathbb{E}_{\mathcal{D}}\left[(f_0 - \hat{f}_0)^2\right] = &\left(f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)^2 \\ &+ \mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)^2\right].\end{aligned} \tag{64}$$

The first term is the squared bias. The bias of the learning procedure at $\mathbf{x}_0$ is

$$\text{Bias}\left[\hat{f}(\mathbf{x}_0)\right] = \mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right] - f(\mathbf{x}_0), \tag{65}$$

so

$$\text{Bias}^2\left[\hat{f}(\mathbf{x}_0)\right] = \left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)] - f(\mathbf{x}_0)\right)^2. \tag{66}$$

The second term is the variance of the fitted prediction:

$$\text{Var}\left[\hat{f}(\mathbf{x}_0)\right] = \mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}_0) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)]\right)^2\right]. \tag{67}$$

Substituting these definitions into Equation 62 gives the bias–variance decomposition:

$$\text{Err}(\mathbf{x}_0) = \sigma^2 + \text{Bias}^2\left[\hat{f}(\mathbf{x}_0)\right] + \text{Var}\left[\hat{f}(\mathbf{x}_0)\right]. \tag{68}$$

Conceptually,

$$\text{Expected Test Error} = \text{Irreducible Error} + \text{Bias}^2 + \text{Variance}. \tag{69}$$

The three components have different interpretations.

5.2.1 Irreducible Error

The term σ^2 represents the variability in the response that remains after conditioning on the predictors available to the model.

Within the assumed data-generating process, this component cannot be reduced by changing the learning algorithm alone.

As discussed previously, however, *irreducible* is relative to the available information. If additional relevant predictors become available, some variability previously contained in the error term may become predictable.

5.2.2 Bias

Bias measures the systematic difference between the average prediction produced by the learning procedure and the underlying relationship:

$$\text{Bias}\left[\hat{f}(\mathbf{x}_0)\right] = \mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right] - f(\mathbf{x}_0).$$

A high-bias model makes similar systematic errors across different training datasets. This usually occurs when the model class is too restrictive to represent the relevant relationship.

High bias is therefore commonly associated with **underfitting**.

5.2.3 Variance

Variance measures how much the fitted prediction changes when the training dataset changes:

$$\mathrm{Var}\left[\hat{f}(\mathbf{x}_0)\right] = \mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}_0) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)]\right)^2\right].$$

A high-variance learning procedure is sensitive to the particular observations included in the training sample.

Small changes in the training data may therefore produce substantially different fitted models.

High variance is commonly associated with **overfitting**.

5.3 Model Complexity and the Bias–Variance Trade-off

Bias and variance are often affected in opposite directions by model flexibility.

A highly constrained model may be unable to represent important structure in the data. Its bias may therefore be high. However, because the model is strongly restricted, different training datasets tend to produce relatively similar fitted models, resulting in low variance.

As model flexibility increases, the model can represent a wider range of relationships. This usually reduces bias.

At the same time, the fitted model can become more sensitive to the particular observations contained in the training set, which tends to increase variance.

A common conceptual pattern is therefore

$$\begin{array}{c} \text{Increasing Model Flexibility} \\ \Downarrow \\ \text{Bias}^2 \downarrow \qquad \text{Variance} \uparrow . \end{array}$$

The expected test error may consequently reach a minimum at an intermediate level of model complexity.

Figure 5 illustrates this behaviour conceptually.

The figure should not be interpreted as a universal quantitative law. The precise shapes of the curves depend on the data-generating process, model class, sample size, regularisation, optimisation procedure, and other characteristics of the learning problem.

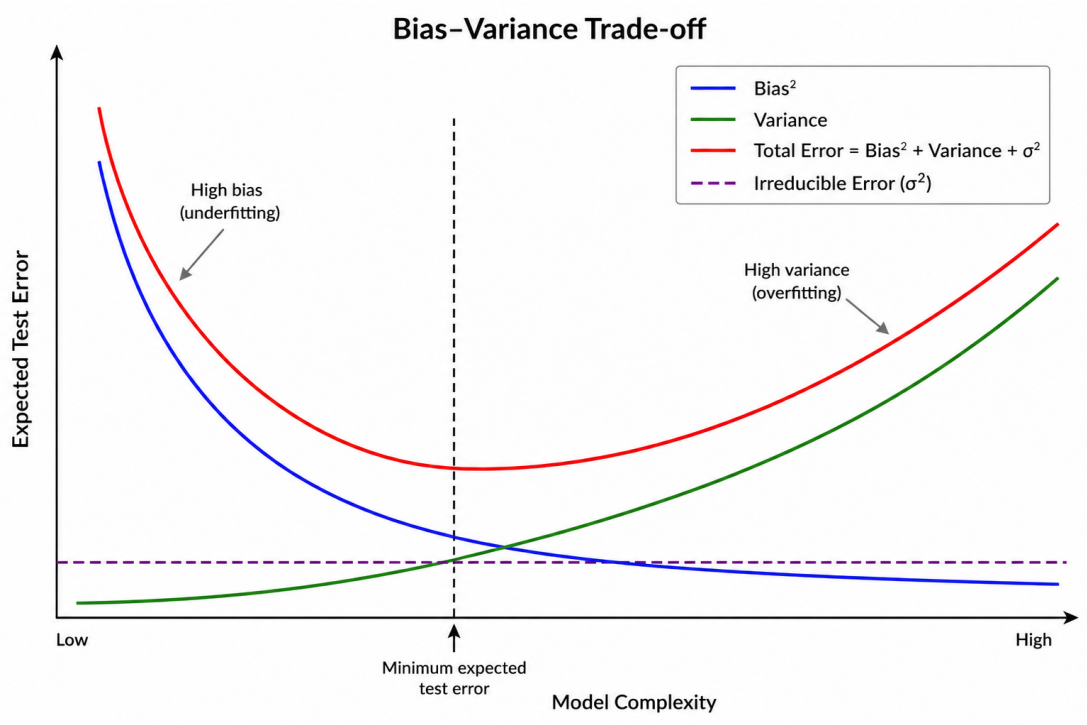

Figure 5: Conceptual representation of the bias–variance trade-off. Increasing model complexity typically reduces squared bias while increasing variance. Expected test error combines irreducible error, squared bias, and variance, and may therefore reach a minimum at an intermediate level of model complexity.

Its purpose is to illustrate the qualitative tension between bias and variance.

The left-hand region corresponds broadly to relatively restrictive models, for which high bias and **underfitting** are more likely. The right-hand region corresponds broadly to highly flexible models, for which increased variance and **overfitting** become more likely.

The minimum of the expected test-error curve represents a useful balance between these competing effects.

It is also important not to equate model complexity simply with the number of parameters.

A model with many parameters may behave in a relatively constrained manner if it is strongly regularised, while a model with fewer explicit parameters may still be highly flexible. Effective complexity depends on the model class, architecture, regularisation, feature representation, sample size, training procedure, and other restrictions imposed on the learning process.

The practical objective is therefore not to maximise or minimise complexity in isolation, but to choose a degree of flexibility that supports reliable generalisation.

5.4 Training Error and Test Error

The distinction between training performance and generalisation performance is fundamental.

Let the training dataset be

$$\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N_{\text{train}}}.$$

Once the model has been fitted, the **training error** measures its performance on the same observations used during training.

For a regression problem using mean squared error,

$$\text{MSE}_{\text{train}} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \left[y_i - \hat{f}(\mathbf{x}_i) \right]^2. \tag{70}$$

The training error is useful for understanding how well the model fits the observed sample, but it is generally an optimistic measure of future predictive performance because the model parameters were chosen using those same observations.

What we ultimately want to estimate is the **test error**, or prediction error: the expected error obtained when the fitted model is applied to new observations that were not used during training.

If a sufficiently large independent test dataset is available,

$$\mathcal{D}_{\text{test}} = \left\{(\mathbf{x}_j, y_j)\right\}_{j=1}^{N_{\text{test}}},$$

the test error can be estimated directly. For squared-error regression,

$$\text{MSE}_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{j=1}^{N_{\text{test}}} \left[y_j - \hat{f}(\mathbf{x}_j)\right]^2. \tag{71}$$

In practice, however, data are often limited, and allocating a large fraction of the available observations to an independent test set may be inefficient.

This motivates methods that estimate generalisation performance using the available training data more effectively.

5.5 The Validation Set Approach

The simplest approach is to divide the available data randomly into two subsets:

- a **training set**, used to fit the model;
- a **validation set**, used to estimate predictive performance.

Suppose that

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{validation}},$$

with the two subsets containing different observations.

The model is fitted using $\mathcal{D}_{\text{train}}$ and then evaluated on $\mathcal{D}_{\text{validation}}$.

For squared-error regression, the validation error may be estimated as

$$\text{MSE}_{\text{validation}} = \frac{1}{N_{\text{validation}}} \sum_{i \in \mathcal{D}_{\text{validation}}} \left[y_i - \hat{f}_{\text{train}}(\mathbf{x}_i)\right]^2. \tag{72}$$

This approach is simple and computationally inexpensive, but it has two important limitations.

First, the estimated validation error can be highly dependent on the particular random split. Different observations in the training and validation sets may produce noticeably different estimates of predictive performance.

Second, only a fraction of the available data are used to fit the model. A model trained on the complete dataset would generally have access to more information and may perform better than a model trained on the reduced training subset. The validation error can therefore provide a somewhat pessimistic estimate of the performance of a model ultimately trained on all available data.

Cross-validation addresses these limitations by repeating the training-validation process across different subsets of the data.

5.6 Cross-Validation

Cross-validation is a resampling procedure designed primarily to estimate how well a learning method is expected to perform on unseen observations and to support decisions about model selection and tuning.

Rather than relying on a single train-validation split, cross-validation repeatedly changes which observations are used for training and which are used for validation.

Two important forms are K**-fold cross-validation** and **Leave-One-Out Cross-Validation** (**LOOCV**).

5.6.1 K-Fold Cross-Validation

In K**-fold cross-validation**, the available training data are divided into K approximately equal, non-overlapping subsets or folds:

$$\mathcal{D} = \mathcal{D}^{(1)} \cup \mathcal{D}^{(2)} \cup \cdots \cup \mathcal{D}^{(K)}.$$

The procedure consists of K iterations.

At iteration k, fold $\mathcal{D}^{(k)}$ is used for validation, while the remaining $K - 1$ folds are used for training.

Thus,

$$\mathcal{D}_{\text{train}}^{(-k)} = \mathcal{D} \setminus \mathcal{D}^{(k)}.$$

Let $\widehat{f}^{(-k)}$ denote the model fitted without fold k.

For a regression problem using squared error, the validation error for fold k is

$$\mathrm{MSE}^{(k)} = \frac{1}{N_k} \sum_{i \in \mathcal{D}^{(k)}} \left[y_i - \hat{f}^{(-k)}(\mathbf{x}_i) \right]^2, \tag{73}$$

where N_k is the number of observations in the k-th fold.

The cross-validation estimate is then obtained by averaging the error across all folds:

$$\mathrm{CV}_K = \frac{1}{K} \sum_{k=1}^{K} \mathrm{MSE}^{(k)}. \tag{74}$$

In this way, every observation is used for validation exactly once and for training in the remaining $K - 1$ fits.

Figure 6 illustrates the procedure for $K = 5$.

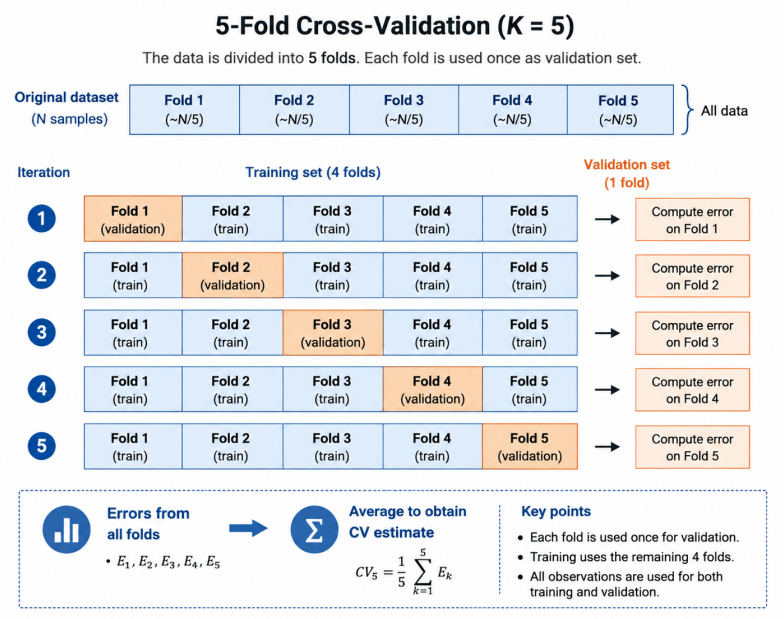

Figure 6: Five-fold cross-validation. At each iteration, one fold is reserved for validation while the remaining folds are used for training. Each fold serves as the validation set exactly once.

The choice of K involves a practical trade-off.

With a small value of K, each model is trained on a smaller fraction of the available data, which may introduce more bias into the estimate of performance.

As K increases, each fitted model uses a larger fraction of the data. However, the different training sets become increasingly similar to one another, and the resulting validation estimates may become more correlated.

Values such as $K = 5$ or $K = 10$ are widely used in practice because they often provide a reasonable balance between computational cost and estimation quality, although there is no universally optimal value of K.

Cross-validation is particularly useful for **model selection** and **hyperparameter tuning**.

Suppose, for example, that a learning method depends on a hyperparameter λ. For several candidate values, we can compute

$$\mathrm{CV}_K(\lambda)$$

and choose

$$\hat{\lambda} = \arg\min_{\lambda} \mathrm{CV}_K(\lambda). \tag{75}$$

Once the modelling choices have been made, the final selected model can be fitted using the chosen configuration.

5.6.2 Leave-One-Out Cross-Validation

Leave-One-Out Cross-Validation (LOOCV) is the limiting case of K-fold cross-validation in which

$$K = N.$$

Each validation fold therefore contains a single observation.

For observation i, the model is fitted using all observations except $(\mathbf{x}_i, y_i)$. Denote this fitted model by $\hat{f}^{(-i)}$.

The prediction error for the omitted observation is

$$\left[y_i - \hat{f}^{(-i)}(\mathbf{x}_i)\right]^2.$$

The LOOCV estimate for squared-error regression is therefore

$$\mathrm{CV}_{\mathrm{LOO}} = \frac{1}{N} \sum_{i=1}^{N} \left[y_i - \hat{f}^{(-i)}(\mathbf{x}_i)\right]^2. \tag{76}$$

LOOCV has the advantage that each fitted model uses $N - 1$ observations, so the training datasets are almost as large as the complete dataset.

However, the method has two practical drawbacks.

First, it can be computationally expensive because the model may need to be fitted N times.

Second, the training sets used in different iterations are extremely similar. The resulting error estimates can therefore be strongly correlated, which can increase the variance of the overall cross-validation estimate relative to moderate values of K.

For these reasons, K-fold cross-validation with values such as 5 or 10 is often preferred in practical machine-learning workflows.

5.7 Model Selection and Model Evaluation

Cross-validation also makes it important to distinguish **model selection** from **model evaluation**.

Model selection concerns decisions made while constructing the model, such as:

- which model class to use;
- which predictors or representations to include;
- how much regularisation to apply;
- which hyperparameters to choose;
- how much model complexity to allow.

These decisions may be guided by cross-validation or by a dedicated validation set.

Model evaluation asks a different question:

> **How well does the final selected modelling procedure perform on genuinely unseen data?**

For this reason, when sufficient data are available, an independent **test set** should be kept separate from the model-selection process.

Repeatedly consulting the test set while choosing models or tuning hyperparameters gradually incorporates information from the test observations into the modelling process. The resulting test performance can then become optimistically biased.

A useful workflow is therefore

Training Data $\longrightarrow$ Model Fitting and Selection

$\Downarrow$

Final Selected Model $\longrightarrow$ Independent Test Evaluation.

Cross-validation is primarily a tool for model selection and estimation of out-of-sample performance during model development. A final test set, when available, provides an independent assessment after those modelling decisions have been completed.

5.8 Bootstrap Method

Cross-validation is primarily concerned with predictive performance. A related but different problem is estimating the **uncertainty or sampling variability** of a statistic, parameter estimate, prediction, or fitted model.

This is one of the main purposes of the **bootstrap**.

Suppose that we are interested in estimating an unknown quantity θ using an estimator $\hat{\theta}$ calculated from the observed training data.

After obtaining $\hat{\theta}$, a natural question is:

> **How much would this estimate change if we obtained a different sample from the same underlying population?**

For simple estimators, this sampling variability may sometimes be derived analytically. For more complicated estimators or learning procedures, analytical derivations may be difficult or unavailable.

The bootstrap provides a computational approach to approximating this variability from the observed data themselves.

Let the training dataset be

$$\mathcal{D} = (d_1, d_2, \dots, d_N),$$

where, in a supervised-learning problem, each observation may be written as $d_i = (\mathbf{x}_i, y_i)$.

A bootstrap sample $\mathcal{D}^{*b}$ is obtained by drawing N observations **with replacement** from $\mathcal{D}$:

$$\mathcal{D}^{*b} = (d_1^{*b}, d_2^{*b}, \dots, d_N^{*b}),$$

for

$$b = 1, 2, \ldots, B.$$

Because sampling is performed with replacement, an original observation may appear several times in a bootstrap sample, while another observation may not appear at all.

The process is repeated B times, generating

$$\mathcal{D}^{*1}, \mathcal{D}^{*2}, \ldots, \mathcal{D}^{*B}.$$

Suppose that $\hat{\theta}$ is the quantity calculated from the original dataset. Applying the same estimation procedure to each bootstrap sample produces the bootstrap replicates

$$\hat{\theta}^{*1}, \hat{\theta}^{*2}, \ldots, \hat{\theta}^{*B}.$$

Figure 7 provides a schematic representation of this procedure.

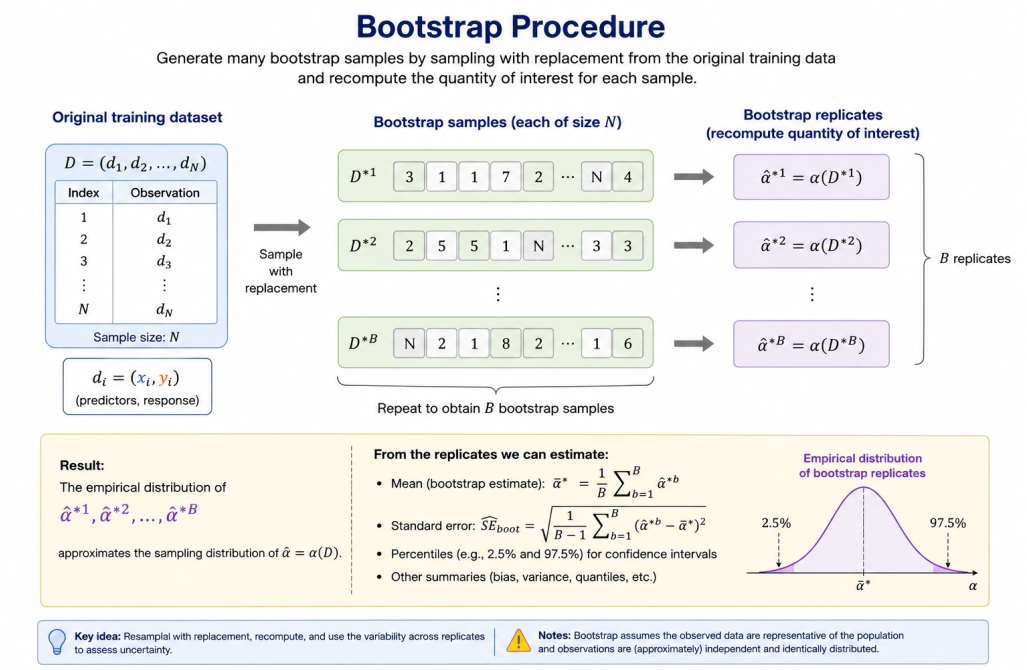

Figure 7: Bootstrap procedure. Repeated samples of the same size as the original training dataset are drawn with replacement, and the quantity of interest is recalculated for each bootstrap sample.

The empirical distribution of the bootstrap replicates approximates the sampling distribution of the estimator.

For example, the mean of the bootstrap replicates is

$$\bar{\hat{\theta}}^{*} = \frac{1}{B}\sum_{b=1}^{B}\hat{\theta}^{*b}, \tag{77}$$

and the bootstrap estimate of the standard error is

$$\widehat{\mathrm{SE}}_{\mathrm{boot}}(\hat{\theta}) = \sqrt{\frac{1}{B-1}\sum_{b=1}^{B}\left(\hat{\theta}^{*b} - \bar{\hat{\theta}}^{*}\right)^{2}}. \tag{78}$$

The bootstrap can therefore be used to study the variability of parameter estimates, model predictions, feature importance measures, and other quantities whose sampling behaviour is difficult to obtain analytically.

It can also be used to construct confidence intervals and to study the stability of a learning procedure.

5.9 Cross-Validation and Bootstrap

Cross-validation and bootstrap are both **resampling methods**, and both involve repeatedly working with modified versions of the observed dataset.

However, their main purposes are different.

Cross-validation asks primarily:

> **How well is this learning procedure expected to predict new observations?**

Bootstrap asks primarily:

> **How variable or uncertain is this estimated quantity across repeated samples?**

Thus,

- **Cross-validation** is primarily used for estimating out-of-sample predictive performance, model comparison, and hyperparameter selection.
- **Bootstrap** is primarily used for estimating sampling variability, uncertainty, and stability.

The two procedures are therefore complementary rather than interchangeable.

5.10 What Machine Learning Can and Cannot Do

The discussion above also clarifies some important limitations of machine-learning models.

Machine learning can identify statistical structure in the information made available to it. It can exploit complex relationships among predictors, learn useful representations, and provide predictions in situations where an explicit analytical model may be unavailable or impractical.

However, a model cannot reliably learn information that is absent from the data.

If relevant predictors are not available, if the training data are systematically biased, if the sample is not representative of the population in which the model will be deployed, or if the data-generating process changes substantially over time, predictive performance may deteriorate even when the training procedure itself is technically correct.

The quality of a machine-learning model therefore depends not only on the algorithm but also on the quality and relevance of the data on which it was trained.

A second important limitation concerns the distinction between **prediction** and **causality**.

A predictive model may discover that two variables are strongly associated and use that relationship effectively for prediction. This does not imply that changing one of those variables would cause the other to change.

The question

> *What is likely to happen?*

is not the same as

> *What would happen if we intervened?*

The latter is a causal question and generally requires additional assumptions, experimental design, or specialised causal-inference methods.

A third limitation follows from the distinction between training and deployment environments.

Machine-learning models are evaluated under some assumed relationship between the training data and future observations. If that relationship changes — for example because customer behaviour, market conditions, fraud strategies, regulations, measurement processes, or operational systems change — past predictive performance may no longer accurately describe future performance.

This phenomenon is commonly referred to as **distribution shift** or, in some contexts, **concept drift**.

Consequently, machine-learning performance is always conditional on the context in which the model is trained and used.

In practice, reliable modelling depends on several factors:

- the quality and representativeness of the data;
- the predictors available at prediction time;
- the assumptions and inductive biases of the chosen model class;
- the learning objective;
- the model-selection procedure;
- the evaluation strategy;
- and the stability of the environment in which the model will operate.

Machine learning should therefore not be understood as a procedure that extracts an unconstrained truth directly from data. It learns useful relationships or representations within the information, assumptions, and objectives supplied to the learning process.

The appropriate standard is not whether a model is sophisticated, fashionable, or capable of fitting the training data particularly closely.

The appropriate standard is whether it provides **reliable, useful, and properly validated performance for the problem it is intended to solve**.

6 Problems

The following problems are intended to reinforce the main ideas developed in these notes. They combine mathematical derivations, small numerical exercises, and conceptual questions that require reasoning about a modelling problem rather than simply recalling definitions.

The emphasis is on understanding the assumptions behind a method, interpreting mathematical results, and explaining the consequences of modelling decisions.

6.1 Problem 1 — Predictor and Response Notation

Suppose a supervised-learning dataset contains $N = 1{,}000$ observations and $M = 8$ predictors.

6.1.1 Questions

1. What are the dimensions of the predictor matrix $\mathbf{X}$?
2. What are the dimensions of the response vector $\mathbf{y}$?
3. What does $\mathbf{x}_i$ represent, and what are its dimensions?
4. What does x_{ij} represent?
5. What does the j-th column of $\mathbf{X}$ represent?
6. Write the dataset using the notation $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$.

6.1.2 Solution

Since there are $N = 1{,}000$ observations and $M = 8$ predictors, the predictor matrix contains one row for each observation and one column for each predictor. Therefore,

$$\mathbf{X} \in \mathbb{R}^{1000 \times 8}.$$

Explicitly,

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{18} \\ x_{21} & x_{22} & \cdots & x_{28} \\ \vdots & \vdots & \ddots & \vdots \\ x_{1000,1} & x_{1000,2} & \cdots & x_{1000,8} \end{pmatrix}.$$

The response vector contains one response for each observation, so

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{1000} \end{pmatrix} \in \mathbb{R}^{1000}.$$

The vector $\mathbf{x}_i$ represents all predictor values associated with observation i:

$$\mathbf{x}_i = \begin{pmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{i8} \end{pmatrix} \in \mathbb{R}^{8}.$$

Because observations are stored as rows in $\mathbf{X}$, the i-th row of the matrix is $\mathbf{x}_i^\top$:

$$\mathbf{x}_i^\top = (x_{i1}, x_{i2}, \dots, x_{i8}).$$

The scalar x_{ij} represents the value of predictor j for observation i.

Consequently, the j-th column,

$$\begin{pmatrix} x_{1j} \\ x_{2j} \\ \vdots \\ x_{1000,j} \end{pmatrix},$$

contains the values of predictor j across all observations.

Finally, the supervised-learning dataset can be written as

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{1000}.$$

An important distinction is therefore

$$\mathbf{x}_i \in \mathbb{R}^M, \qquad \mathbf{X} \in \mathbb{R}^{N \times M}, \qquad \mathbf{y} \in \mathbb{R}^N.$$

Here, $\mathbf{x}_i$ describes one observation, $\mathbf{X}$ describes the predictor information for the complete dataset, and $\mathbf{y}$ contains the corresponding responses.

6.2 Problem 2 — Identifying the Learning Problem

Consider the following modelling problems:

1. Predicting the selling price of a house from its size, location, age, and number of bedrooms.
2. Determining whether a credit-card transaction is legitimate or fraudulent.
3. Grouping customers according to purchasing behaviour when no customer segments have been defined beforehand.
4. Reducing a dataset containing 200 correlated measurements to a smaller number of variables that retain most of its variation.
5. Teaching an autonomous agent to choose actions in an environment where each action produces a reward and changes the subsequent state.
6. Predicting the amount of electricity that a household will consume tomorrow.

For each case:

1. Determine whether the problem is primarily supervised learning, unsupervised learning, or reinforcement learning.
2. For supervised-learning problems, determine whether the task is regression or classification.
3. Explain the reasoning behind the classification.

6.2.1 Solution

Case 1: House-price prediction

The response is the selling price of the house, which is a numerical quantity. Predictor-response pairs are available.

Therefore, this is a **supervised regression problem**.

The objective is to learn a mapping of the form

$$\mathbf{x} \longrightarrow y \in \mathbb{R},$$

where $\mathbf{x}$ contains the characteristics of a house and y represents its selling price.

Case 2: Credit-card fraud detection

Each transaction is associated with a response indicating whether it is legitimate or fraudulent.

The response therefore represents membership in one of two classes.

This is a **supervised classification problem**, specifically binary classification.

For example,

$$y = \begin{cases} 0, & \text{legitimate,} \\ 1, & \text{fraudulent.} \end{cases}$$

Case 3: Customer segmentation

No predefined response or customer segment is available. The objective is to discover groups from similarities and differences among customer observations.

This is an **unsupervised-learning problem**, specifically clustering.

The important distinction from classification is that the groups are not supplied to the algorithm as known responses. They must be inferred from the structure of the data.

Case 4: Dimensionality reduction

There is no response to predict. Instead, the objective is to construct a lower-dimensional representation of the predictor data while retaining relevant structure.

This is an **unsupervised-learning problem**, specifically dimensionality reduction.

Principal Component Analysis would be a classical example of a method that could be considered for such a problem.

Case 5: Sequential decision-making

The learner chooses actions, observes their consequences, receives rewards, and subsequently encounters new states.

This is a **reinforcement-learning problem**.

Unlike supervised learning, the learner is not provided with the correct action for every possible state. It learns a policy from the consequences of its interaction with the environment.

Case 6: Electricity-consumption prediction

The response is the amount of electricity consumed, which is numerical.

Assuming historical predictor-response observations are available, this is a **supervised regression problem**.

The six examples illustrate that the appropriate learning paradigm depends first on the structure of the problem and on what information is available during learning, rather than on the name of a particular algorithm.

6.3 Problem 3 — Regression or Classification Is Not Determined by the Predictors

A bank has a dataset containing information about its customers:

$$\mathbf{x}_i = (\text{income}, \text{age}, \text{account balance}, \text{number of transactions}, \ldots)^\top.$$

The bank considers two modelling objectives.

Model A: Predict the amount of money that a customer will spend during the next month.

Model B: Predict whether the customer will close their account during the next month.

6.3.1 Questions

1. Is Model A a regression or classification problem?
2. Is Model B a regression or classification problem?
3. Both models use essentially the same predictors. Why do they correspond to different learning problems?
4. Suppose Model B estimates

$$P(Y = 1 \mid \mathbf{X} = \mathbf{x}) = 0.73.$$

Does the numerical output 0.73 make Model B a regression model?

6.3.2 Solution

Model A predicts an amount of money. Since the response is a numerical quantity, Model A is a **regression model**.

We can represent its response as

$$Y \in \mathbb{R}.$$

Model B predicts whether a customer belongs to one of two classes:

$$Y = \begin{cases} 0, & \text{customer remains,} \\ 1, & \text{customer leaves.} \end{cases}$$

It is therefore a **classification model**.

The fact that both problems use the same predictors does not determine the type of supervised-learning problem. The distinction between regression and classification is determined primarily by the nature of the response and the objective of the prediction.

This also explains why the answer to Question 4 is **no**.

A classifier can produce a numerical quantity internally. For example,

$$\hat{p}(\mathbf{x}) = P(Y = 1 \mid \mathbf{X} = \mathbf{x}) = 0.73.$$

The quantity 0.73 represents an estimated probability of class membership.

A decision rule may subsequently convert this probability into a class prediction. If the threshold is $c = 0.5$, then

$$0.73 > 0.5$$

and the predicted class would be

$$\hat{y} = 1.$$

The fact that the model estimates a numerical probability does not transform the original learning objective into regression. The target remains class membership.

6.3.3 Follow-up Question

Suppose the cost of failing to identify a customer who is about to leave is substantially greater than the cost of incorrectly flagging a customer who would have stayed. Should the classification threshold necessarily remain at 0.5?

6.3.4 Answer

No.

The threshold should reflect the purpose of the model and the relative consequences of false positives and false negatives.

Reducing the threshold would generally classify more customers as likely to leave. This tends to increase sensitivity to potential churners but also increases the number of false positives.

The appropriate threshold is therefore a decision problem and cannot be determined from the estimated probabilities alone.

6.4 Problem 4 — Interpreting the Statistical Learning Model

Suppose the response is described by

$$Y = f(\mathbf{X}) + \varepsilon,$$

with

$$\mathbb{E}[\varepsilon \mid \mathbf{X}] = 0.$$

6.4.1 Questions

1. What does $f(\mathbf{X})$ represent?
2. What does ε represent?
3. Show that

$$\mathbb{E}[Y \mid \mathbf{X}] = f(\mathbf{X}).$$

4. Does ε necessarily represent pure physical randomness?
5. Suppose an important predictor is omitted from $\mathbf{X}$. Where may its effect appear in this representation?
6. If that omitted predictor later becomes available, is all of the previous irreducible error necessarily still irreducible?

6.4.2 Solution

The function $f(\mathbf{X})$ represents the systematic component of the relationship between the available predictors and the response.

The term ε represents the component of the response that remains unexplained after conditioning on the information contained in $\mathbf{X}$.

To derive the conditional expectation, start from

$$Y = f(\mathbf{X}) + \varepsilon.$$

Taking conditional expectation given $\mathbf{X}$,

$$\begin{aligned}\mathbb{E}[Y \mid \mathbf{X}] &= \mathbb{E}[f(\mathbf{X}) + \varepsilon \mid \mathbf{X}] \\ &= \mathbb{E}[f(\mathbf{X}) \mid \mathbf{X}] + \mathbb{E}[\varepsilon \mid \mathbf{X}].\end{aligned}$$

Since $f(\mathbf{X})$ is completely determined once $\mathbf{X}$ is known,

$$\mathbb{E}[f(\mathbf{X}) \mid \mathbf{X}] = f(\mathbf{X}).$$

By assumption,

$$\mathbb{E}[\varepsilon \mid \mathbf{X}] = 0.$$

Therefore,

$$\mathbb{E}[Y \mid \mathbf{X}] = f(\mathbf{X}).$$

The error term should not automatically be interpreted as pure physical randomness.

It may contain several sources of unexplained variation, including measurement error, genuinely stochastic effects, variables that cannot be measured, and relevant predictors that were simply not included in the model.

Suppose, for example, that the true response depends on both $\mathbf{X}$ and another variable Z:

$$Y = g(\mathbf{X}, Z) + \eta,$$

but Z is unavailable.

If we construct a model using only $\mathbf{X}$, part of the variation associated with Z may appear in the unexplained component of the reduced representation.

If Z later becomes available and contains predictive information not already contained in $\mathbf{X}$, some of the variation previously treated as irreducible may become reducible.

Thus, **irreducible error is irreducible relative to the information available to the model**, not necessarily in an absolute sense.

6.5 Problem 5 — Reducible and Irreducible Error

At a particular test point $\mathbf{x}_0$, suppose

$$Y_0 = f(\mathbf{x}_0) + \varepsilon_0,$$

where

$$\mathbb{E}[\varepsilon_0] = 0$$

and

$$\mathrm{Var}(\varepsilon_0) = 4.$$

A fitted model gives

$$f(\mathbf{x}_0) = 12$$

and

$$\hat{f}(\mathbf{x}_0) = 10.5.$$

Assume for this problem that the fitted function is fixed.

6.5.1 Questions

1. Calculate the reducible component of the expected squared prediction error.
2. Calculate the irreducible component.
3. Calculate the total expected squared prediction error at $\mathbf{x}_0$.
4. Derive the result rather than applying the decomposition directly.
5. If a better model produces $\hat{f}(\mathbf{x}_0) = 11.8$, calculate the new expected squared prediction error.
6. Can improving $\hat{f}$ reduce the expected error below 4 under the assumptions of the problem?

6.5.2 Solution

The expected squared prediction error is

$$\mathbb{E}\left[\left(Y_0 - \hat{f}(\mathbf{x}_0)\right)^2\right].$$

Substituting

$$Y_0 = f(\mathbf{x}_0) + \varepsilon_0,$$

we obtain

$$\mathbb{E}\left[\left(Y_0 - \hat{f}(\mathbf{x}_0)\right)^2\right] = \mathbb{E}\left[\left(f(\mathbf{x}_0) + \varepsilon_0 - \hat{f}(\mathbf{x}_0)\right)^2\right].$$

Using the numerical values,

$$f(\mathbf{x}_0) - \hat{f}(\mathbf{x}_0) = 12 - 10.5 = 1.5.$$

Therefore,

$$\begin{aligned}\mathbb{E}\left[(1.5+\varepsilon_0)^2\right] &= \mathbb{E}\left[2.25+3\varepsilon_0+\varepsilon_0^2\right] \\ &= 2.25+3\mathbb{E}[\varepsilon_0]+\mathbb{E}[\varepsilon_0^2].\end{aligned}$$

Since

$$\mathbb{E}[\varepsilon_0]=0,$$

the middle term vanishes.

Furthermore,

$$\text{Var}(\varepsilon_0)=\mathbb{E}[\varepsilon_0^2]-\mathbb{E}[\varepsilon_0]^2.$$

Since the mean is zero,

$$\mathbb{E}[\varepsilon_0^2]=\text{Var}(\varepsilon_0)=4.$$

Hence,

$$\mathbb{E}\left[\left(Y_0-\hat{f}(\mathbf{x}_0)\right)^2\right]=2.25+4=6.25.$$

The **reducible error** is therefore

$$(12-10.5)^2=2.25,$$

while the **irreducible error** is

$$\sigma^2=4.$$

The total expected squared prediction error is

$$6.25.$$

Now suppose that the improved model gives

$$\hat{f}(\mathbf{x}_0)=11.8.$$

The reducible component becomes

$$(12 - 11.8)^2 = 0.2^2 = 0.04.$$

Therefore,

$$\text{Err}(\mathbf{x}_0) = 0.04 + 4 = 4.04.$$

The expected prediction error has fallen from 6.25 to 4.04 because the estimate of the systematic relationship has improved.

However, even if the model recovered the underlying function exactly at this point,

$$\hat{f}(\mathbf{x}_0) = f(\mathbf{x}_0),$$

the expected squared error would still be

$$\text{Err}(\mathbf{x}_0) = 0 + \sigma^2 = 4.$$

Under the assumptions of this problem, 4 is therefore the lower bound imposed by the irreducible error.

6.6 Problem 6 — What Does Model Fitting Actually Mean?

Suppose a regression model is defined by

$$f_\theta(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2,$$

with

$$\theta = (\theta_0, \theta_1, \theta_2)^\top.$$

The model is fitted by solving

$$\hat{\theta} = \arg\min_\theta \sum_{i=1}^{N} \left[y_i - f_\theta(\mathbf{x}_i) \right]^2 .$$

6.6.1 Questions

1. What does arg min mean?
2. What is being minimised?
3. What does $\widehat{\theta}$ represent?
4. Is the result of the optimisation the minimum value of the loss function or the parameter vector at which that minimum occurs?
5. Once $\widehat{\theta}$ has been found, write the fitted function $\widehat{f}$.
6. Why can fitting the model more closely to the training observations eventually become undesirable?

6.6.2 Solution

The expression

$$\arg\min_{\theta} \mathcal{L}(\theta)$$

means: find the value of θ at which the objective function $\mathcal{L}$ reaches its minimum.

This distinction is important.

The expression

$$\min_{\theta} \mathcal{L}(\theta)$$

refers to the **minimum value of the objective function**, whereas

$$\arg\min_{\theta} \mathcal{L}(\theta)$$

refers to the **parameter values that produce that minimum**.

In this problem, the objective function is

$$\mathcal{L}(\theta) = \sum_{i=1}^{N} \left[y_i - f_\theta(\mathbf{x}_i) \right]^2 .$$

For the specified linear model,

$$\mathcal{L}(\theta_0, \theta_1, \theta_2) = \sum_{i=1}^{N} \left[y_i - (\theta_0 + \theta_1 x_{i1} + \theta_2 x_{i2}) \right]^2 .$$

The fitting procedure searches for the parameter vector

$$\hat{\theta} = (\hat{\theta}_0, \hat{\theta}_1, \hat{\theta}_2)^\top$$

that minimises this quantity.

Once these parameters have been estimated, the fitted model is

$$\hat{f}(\mathbf{x}) = \hat{\theta}_0 + \hat{\theta}_1 x_1 + \hat{\theta}_2 x_2.$$

The distinction between the model class and the fitted model is worth making explicit.

Before training,

$$f_\theta$$

represents an entire family of possible functions indexed by θ.

After training,

$$f_{\hat{\theta}}$$

is the particular member of that family selected by the learning procedure.

However, minimising training error is not itself the final objective of predictive modelling.

If the model class becomes sufficiently flexible, it may begin to reproduce noise and sample-specific fluctuations rather than only the systematic relationship of interest. Training error can continue to decrease while performance on new observations begins to deteriorate.

This is the basic mechanism underlying overfitting.

6.7 Problem 7 — Does a Better Training Fit Mean a Better Model?

Two regression models are fitted to the same training dataset.

Their mean squared errors are:

Model	Training MSE	Validation MSE
A	1.8	9.4
B	5.1	5.8

6.7.1 Questions

1. Which model fits the training data more closely?
2. Which model currently provides stronger evidence of generalisation?
3. What does the difference between the training and validation errors suggest about Model A?
4. What does the result suggest about Model B?
5. Can we conclude from these four numbers alone that Model B is definitively the better model?
6. What additional information would you want before making a final modelling decision?

6.7.2 Solution

Model A has the smaller training error:

$$1.8 < 5.1.$$

It therefore fits the training observations more closely.

However, Model A has a validation error of 9.4, while Model B has a validation error of 5.8:

$$5.8 < 9.4.$$

Based on this validation split, Model B provides stronger evidence of performance on observations that were not used for fitting.

The gap between training and validation error for Model A is

$$9.4 - 1.8 = 7.6.$$

For Model B, the corresponding gap is

$$5.8 - 5.1 = 0.7.$$

The large discrepancy for Model A is consistent with **overfitting**: the model fits the training observations very closely but appears not to transfer that performance to the validation data.

Model B has a higher training error but substantially better validation performance. This suggests that it may generalise better.

However, it would be premature to conclude from these numbers alone that Model B is definitively superior.

The validation errors come from a particular set of observations. If the dataset is small, a different train-validation split could produce noticeably different results.

Before making a final decision, useful questions include:

- How large are the training and validation sets?
- How were they sampled?
- Are the observations independent?
- Is the validation set representative of the population in which the model will be used?
- Are the differences stable across different splits?
- What metric is appropriate for the actual objective?
- Were any modelling decisions made after inspecting the validation results?
- Is there any possibility of data leakage?
- How complex are the two models?
- Are there constraints on interpretability, computational cost, latency, or deployment?
- Is an independent test set available for final evaluation?

A natural next step would be to use cross-validation to determine whether the apparent advantage of Model B is stable across different subsets of the available data.

The broader lesson is that **the model with the lowest training error is not necessarily the model with the best predictive performance**. Training error measures fit to observations already seen by the learning procedure; generalisation requires performance on observations that did not participate in fitting.

6.8 Problem 8 — Parametric or Non-Parametric?

Consider the following models:

1. Simple linear regression.
2. A neural network with a fixed architecture containing 2 million trainable parameters.
3. K-Nearest Neighbours.
4. A cubic spline model whose effective complexity can increase as more data become available.
5. A logistic regression model with 50 predictors.

6.8.1 Questions

1. Classify each model as parametric or non-parametric.
2. Does a model become non-parametric simply because it contains a very large number of parameters?
3. Does the absence of an explicit analytical relationship such as

$$f(x) = \theta_0 + \theta_1 x$$

necessarily imply that the model is non-parametric? 4. Why is a neural network still parametric even though we may not know the final explicit functional relationship between predictors and response in advance?

6.8.2 Solution

Simple linear regression is a **parametric model**.

For example,

$$f_\theta(x) = \theta_0 + \theta_1 x$$

is completely specified by the finite-dimensional parameter vector

$$\theta = (\theta_0, \theta_1)^\top.$$

Once these parameters have been estimated, the fitted model is completely determined.

A neural network with a fixed architecture is also **parametric**.

Even if the network contains millions of trainable parameters, once the architecture is fixed the model can still be represented by a finite-dimensional parameter vector

$$\theta = (\theta_1, \theta_2, ..., \theta_P)^\top,$$

where P may be very large but remains finite.

K-Nearest Neighbours is generally considered **non-parametric**.

The fitted procedure is not reduced to a fixed finite-dimensional parameter vector. Instead, prediction depends directly on the training observations retained by the method.

A spline-based method may also be non-parametric when its effective complexity is allowed to grow with the amount of available data.

Logistic regression with 50 predictors remains **parametric**. It has more parameters than a simple linear model, but the number of parameters is still fixed once the predictor set and model structure are chosen.

The important point is that

> **Parametric does not mean simple, and non-parametric does not mean having no parameters.**

A model does not become non-parametric merely because it contains many parameters.

Similarly, the absence of a simple explicit equation relating the original predictors to the response does not imply that the model is non-parametric.

A neural network may represent an extremely complicated function, but for a fixed architecture it is still determined by a finite vector θ.

Thus, two different questions must be kept separate:

1. **Is the model parametric or non-parametric?**
2. **How flexible is the model class?**

These are related, but they are not equivalent.

6.9 Problem 9 — Functional Form and Model Flexibility

Consider two approaches for modelling an oscillatory dataset.

6.9.1 Model A

The analyst assumes

$$f_\theta(t) = \theta_0 \cos(\theta_1 t + \theta_2).$$

6.9.2 Model B

The analyst uses a neural network

$$f_\theta(t)$$

with several hidden layers and learns all its parameters from the data.

6.9.3 Questions

1. Are both models parametric?
2. Which model has the more strongly prespecified functional form?
3. Which model is generally more flexible?
4. Does Model B make no assumptions?
5. What kinds of assumptions are introduced by the neural network?
6. Which model would you expect to require more data?
7. If Model A is physically correct, why might it generalise better than Model B with limited data?

6.9.4 Solution

Both models are **parametric**.

Model A is determined by the finite-dimensional parameter vector

$$\theta = (\theta_0, \theta_1, \theta_2)^\top.$$

Model B is also determined by a finite-dimensional parameter vector, although that vector may contain a very large number of elements.

The main difference lies in the amount of structure imposed before fitting.

Model A assumes a very specific relationship:

$$f(t) = \theta_0 \cos(\theta_1 t + \theta_2).$$

The analyst has already imposed periodicity and a trigonometric form.

Model B does not prescribe such a simple analytical relationship between time and position. Instead, the network architecture defines a much broader family of possible functions.

Therefore, Model B is generally more flexible.

However, it would be incorrect to say that the neural network makes no assumptions.

Its architecture introduces assumptions through choices such as:

- the number of layers;
- the number of units per layer;
- connectivity;
- activation functions;
- regularisation;
- optimisation procedure;
- parameter initialisation;
- and possibly architectural constraints specific to the problem.

These assumptions form part of the model's **inductive bias**.

Model B would generally be expected to require more data because it has a much larger space of possible relationships to learn from.

If Model A is physically correct, then its strong structural assumptions can be advantageous. The model does not need to learn the general concept of periodicity from the data because that structure is already built into the model class.

With limited data, the stronger prior structure can reduce variance and improve generalisation.

This illustrates an important modelling principle:

> **Flexibility is useful when the underlying relationship is uncertain, but correct structural assumptions can be extremely valuable when reliable prior knowledge is available.**

6.10 Problem 10 — Diagnosing Underfitting and Overfitting

Three models are fitted to the same regression problem.

Model	Training MSE	Validation MSE
A	12.1	12.8
B	4.3	5.0
C	0.7	10.6

6.10.1 Questions

1. Which model appears most likely to underfit?
2. Which model appears most likely to overfit?
3. Which model currently provides the strongest evidence of generalisation?
4. Explain the reasoning rather than selecting a model only from the smallest number in the table.
5. Suppose Model A is made more flexible. What would you generally expect to happen to its training error?
6. Suppose Model C is strongly regularised. What might happen to its training and validation errors?

6.10.2 Solution

Model A has relatively high error on both training and validation data:

$$\text{MSE}_{\text{train}} = 12.1,$$

and

$$\text{MSE}_{\text{validation}} = 12.8.$$

The two values are close, but both are large relative to the other models.

This is consistent with **underfitting**. The model may be too restrictive to capture important structure in the data.

Model C shows the opposite pattern.

Its training error is extremely small:

$$0.7,$$

while its validation error is much larger:

$$10.6.$$

This large generalisation gap suggests **overfitting**. The model appears to fit the training sample very closely but does not transfer that performance to unseen observations.

Model B has

$$\text{MSE}_{\text{train}} = 4.3$$

and

$$\text{MSE}_{\text{validation}} = 5.0.$$

Its validation performance is the best of the three, and the difference between training and validation error is relatively small.

Based on the available information, Model B currently provides the strongest evidence of good generalisation.

If Model A were made more flexible, we would generally expect its training error to decrease because the model would be able to represent a broader set of relationships.

Its validation error might also decrease if the additional flexibility allows the model to capture genuine structure that was previously missed.

However, if flexibility continues to increase, validation error may eventually begin to rise because of overfitting.

For Model C, strong regularisation would generally constrain the fitted model.

Its training error would usually increase because the model would no longer fit the training sample as closely.

At the same time, its validation error may decrease if regularisation reduces sensitivity to noise and sample-specific patterns.

Thus, improving generalisation may sometimes require accepting a worse training fit.

6.11 Problem 11 — Bias and Variance: Conceptual Interpretation

Suppose the same learning algorithm is trained on many different datasets drawn independently from the same population.

At a fixed point $\mathbf{x}_0$, the fitted predictions are:

$$9.8, \quad 10.1, \quad 9.9, \quad 10.0, \quad 10.2.$$

The true underlying value is

$$f(\mathbf{x}_0) = 14.$$

Now consider a second learning algorithm whose predictions are

$$11.0, \quad 17.0, \quad 8.0, \quad 16.0, \quad 13.0.$$

6.11.1 Questions

1. Which algorithm appears to have higher bias?
2. Which algorithm appears to have higher variance?
3. Can an algorithm have low variance and still perform poorly?
4. Can an algorithm have low bias and still perform poorly?
5. Explain why bias and variance measure different kinds of error.

6.11.2 Solution

For the first algorithm, the mean prediction is

$$\frac{9.8 + 10.1 + 9.9 + 10.0 + 10.2}{5} = 10.0.$$

The true value is

$$14.$$

The estimated bias at $\mathbf{x}_0$ is therefore approximately

$$10.0 - 14 = -4.0.$$

The predictions are tightly clustered around 10, so their variance is relatively low.

Thus, the first algorithm exhibits **high bias and low variance**.

For the second algorithm, the mean prediction is

$$\frac{11 + 17 + 8 + 16 + 13}{5} = 13.$$

Its bias is therefore approximately

$$13 - 14 = -1.$$

This is smaller in magnitude than the bias of the first algorithm.

However, the predictions vary substantially from one training dataset to another:

$$8, \quad 11, \quad 13, \quad 16, \quad 17.$$

Thus, the second algorithm exhibits **lower bias but much higher variance**.

This demonstrates why low variance is not sufficient for good predictive performance.

The first algorithm is stable across training datasets, but it is consistently wrong.

Likewise, low bias is not sufficient. A procedure may be correct on average but highly unstable across different samples.

Bias measures **systematic error in the average fitted prediction**.

Variance measures **sensitivity of the fitted prediction to changes in the training sample**.

These are fundamentally different properties.

6.12 Problem 12 — Numerical Bias–Variance Calculation

At a fixed predictor value $\mathbf{x}_0$, suppose the true regression function is

$$f(\mathbf{x}_0) = 8.$$

A learning procedure is repeatedly trained on different datasets. Its predictions at $\mathbf{x}_0$ have

$$\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right] = 7$$

and

$$\mathrm{Var}\left[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right] = 2.5.$$

The irreducible error is

$$\sigma^2 = 1.5.$$

6.12.1 Questions

1. Calculate the bias.
2. Calculate the squared bias.
3. Calculate the variance contribution.
4. Calculate the expected test error at $\mathbf{x}_0$.
5. What fraction of the total error is irreducible?
6. If the variance were reduced to 1.0 without changing the bias, what would the expected test error become?
7. If the bias were eliminated completely but the variance remained 2.5, what would the expected test error become?

6.12.2 Solution

The bias is

$$\begin{aligned}\text{Bias}\left[\hat{f}(\mathbf{x}_0)\right] &= \mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right] - f(\mathbf{x}_0) \\ &= 7 - 8 \\ &= -1.\end{aligned}$$

Therefore, the squared bias is

$$\text{Bias}^2 = (-1)^2 = 1.$$

The variance contribution is given directly as

$$2.5.$$

The bias–variance decomposition gives

$$\text{Err}(\mathbf{x}_0) = \sigma^2 + \text{Bias}^2 + \text{Var}\,.$$

Substituting the values,

$$\begin{aligned}\text{Err}(\mathbf{x}_0) &= 1.5 + 1 + 2.5 \\ &= 5.\end{aligned}$$

The irreducible fraction of the total expected error is

$$\frac{1.5}{5} = 0.3.$$

Thus, 30% of the expected squared prediction error is irreducible under the assumed model.

If the variance were reduced to 1.0 while bias remained unchanged,

$$\text{Err}(\mathbf{x}_0) = 1.5 + 1 + 1 = 3.5.$$

If the bias were completely eliminated while the variance remained 2.5,

$$\text{Err}(\mathbf{x}_0) = 1.5 + 0 + 2.5 = 4.$$

These calculations illustrate that bias and variance both contribute to the reducible part of the expected error.

Improving only one component does not necessarily produce the best possible model if the other component remains large.

6.13 Problem 13 — Deriving the Bias–Variance Decomposition

Suppose that, at a fixed predictor vector $\mathbf{x}_0$,

$$Y_0 = f(\mathbf{x}_0) + \varepsilon_0,$$

where

$$\mathbb{E}[\varepsilon_0] = 0$$

and

$$\text{Var}(\varepsilon_0) = \sigma^2.$$

A learning procedure fitted to a random training dataset $\mathcal{D}$ produces the prediction

$$\hat{f}_{\mathcal{D}}(\mathbf{x}_0).$$

Derive

$$\text{Err}(\mathbf{x}_0) = \sigma^2 + \text{Bias}^2 \left[\hat{f}(\mathbf{x}_0)\right] + \text{Var}\left[\hat{f}(\mathbf{x}_0)\right].$$

Do not quote the result directly. Show the complete derivation.

6.13.1 Solution

The expected squared prediction error is

$$\text{Err}(\mathbf{x}_0) = \mathbb{E}_{\mathcal{D},\varepsilon_0}\left[\left(Y_0 - \hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right)^2\right].$$

Substituting

$$Y_0 = f(\mathbf{x}_0) + \varepsilon_0,$$

we obtain

$$\text{Err}(\mathbf{x}_0) = \mathbb{E}\left[\left(f(\mathbf{x}_0) + \varepsilon_0 - \hat{f}_{\mathcal{D}}(\mathbf{x}_0)\right)^2\right].$$

For compactness, define

$$f_0 = f(\mathbf{x}_0)$$

and

$$\hat{f}_0 = \hat{f}_{\mathcal{D}}(\mathbf{x}_0).$$

Then

$$\text{Err}(\mathbf{x}_0) = \mathbb{E}\left[(f_0 + \varepsilon_0 - \hat{f}_0)^2\right].$$

Rearrange the terms:

$$f_0 + \varepsilon_0 - \hat{f}_0 = (f_0 - \hat{f}_0) + \varepsilon_0.$$

Therefore,

$$\begin{aligned}\mathrm{Err}(\mathbf{x}_0) &= \mathbb{E}\left[\left((f_0 - \hat{f}_0) + \varepsilon_0\right)^2\right] \\ &= \mathbb{E}\left[(f_0 - \hat{f}_0)^2\right] + 2\mathbb{E}\left[(f_0 - \hat{f}_0)\varepsilon_0\right] + \mathbb{E}\left[\varepsilon_0^2\right].\end{aligned}$$

Because the noise associated with the new observation is independent of the fitted model and has zero mean,

$$\begin{aligned}\mathbb{E}\left[(f_0 - \hat{f}_0)\varepsilon_0\right] &= \mathbb{E}[f_0 - \hat{f}_0]\mathbb{E}[\varepsilon_0] \\ &= 0.\end{aligned}$$

Also,

$$\begin{aligned}\mathbb{E}[\varepsilon_0^2] &= \mathrm{Var}(\varepsilon_0) + \mathbb{E}[\varepsilon_0]^2 \\ &= \sigma^2.\end{aligned}$$

Hence,

$$\mathrm{Err}(\mathbf{x}_0) = \mathbb{E}_{\mathcal{D}}\left[(f_0 - \hat{f}_0)^2\right] + \sigma^2.$$

We now decompose the first expectation.

Add and subtract the mean prediction

$$\mathbb{E}_{\mathcal{D}}[\hat{f}_0]$$

inside the difference:

$$f_0 - \hat{f}_0 = f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0] + \mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0.$$

Squaring,

$$\begin{aligned}(f_0 - \hat{f}_0)^2 = &\left(f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)^2 \\ &+ 2\left(f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0\right) \\ &+ \left(\mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0\right)^2.\end{aligned}$$

Taking expectation with respect to $\mathcal{D}$,

$$\begin{aligned}\mathbb{E}_{\mathcal{D}}[(f_0 - \hat{f}_0)^2] = &\left(f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)^2 \\ &+ 2\left(f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)\mathbb{E}_{\mathcal{D}}\left[\mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0\right] \\ &+ \mathbb{E}_{\mathcal{D}}\left[\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0\right)^2\right].\end{aligned}$$

The middle term vanishes because

$$\begin{aligned}\mathbb{E}_{\mathcal{D}}\left[\mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \hat{f}_0\right] &= \mathbb{E}_{\mathcal{D}}[\hat{f}_0] - \mathbb{E}_{\mathcal{D}}[\hat{f}_0] \\ &= 0.\end{aligned}$$

Therefore,

$$\begin{aligned}\mathbb{E}_{\mathcal{D}}[(f_0 - \hat{f}_0)^2] = &\left(f_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)^2 \\ &+ \mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}_0 - \mathbb{E}_{\mathcal{D}}[\hat{f}_0]\right)^2\right].\end{aligned}$$

The first term is the squared bias:

$$\text{Bias}^2\left[\hat{f}(\mathbf{x}_0)\right] = \left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)] - f(\mathbf{x}_0)\right)^2.$$

The second term is the variance:

$$\text{Var}\left[\hat{f}(\mathbf{x}_0)\right] = \mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}_0) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)]\right)^2\right].$$

Thus,

$$\text{Err}(\mathbf{x}_0) = \sigma^2 + \text{Bias}^2\left[\hat{f}(\mathbf{x}_0)\right] + \text{Var}\left[\hat{f}(\mathbf{x}_0)\right].$$

The decomposition separates the expected test error into:

- irreducible variability σ^2;
- systematic error represented by squared bias;
- instability across training samples represented by variance.

6.14 Problem 14 — Model Complexity Is Not Just Parameter Count

Consider the following statement:

> *Model A has 100,000 parameters and Model B has 500 parameters. Therefore Model A is necessarily more complex and more prone to overfitting.*

6.14.1 Questions

1. Is the statement necessarily correct?
2. Why is parameter count an incomplete measure of model complexity?
3. How can regularisation alter effective model complexity?
4. Can a model with many parameters generalise well?
5. Give examples of other factors that influence effective complexity.
6. How does this relate to the bias–variance trade-off?

6.14.2 Solution

The statement is **not necessarily correct**.

The number of parameters provides some information about the size of a model, but it does not uniquely determine its **effective complexity**.

Suppose Model A contains many parameters but is subject to strong regularisation. The optimisation procedure may constrain those parameters so that only a relatively limited family of effective functions can be learned.

Model B may contain fewer explicit parameters but still be capable of representing highly variable relationships over the relevant predictor space.

Regularisation illustrates this directly.

Suppose the learning objective is

$$\hat{\theta} = \arg\min_{\theta} \left[\mathcal{L}(\theta) + \lambda \Omega(\theta) \right],$$

where $\Omega(\theta)$ penalises some notion of model complexity.

As λ increases, the regularisation penalty becomes more influential.

The model may then become effectively less flexible even though the number of parameters has not changed.

Thus, the same architecture can behave differently depending on the strength of regularisation.

A model with many parameters can therefore generalise well when its effective flexibility is appropriately controlled and when sufficient informative data are available.

Other factors influencing effective complexity include:

- architecture;
- regularisation;
- constraints on parameters;
- feature representation;
- sample size;
- optimisation procedure;
- early stopping;
- tree depth;
- neighbourhood size;
- smoothing parameters;
- and the geometry of the model class itself.

This connects directly to the bias–variance trade-off.

Increasing effective flexibility often reduces bias because the model can represent a broader range of relationships.

However, excessive flexibility can increase variance because the fitted model becomes more sensitive to sample-specific fluctuations.

The relevant question is therefore not simply

> *How many parameters does the model have?*

but rather

> **How flexible is the learning procedure relative to the amount and structure of the available data?**

That is the more useful way to reason about model complexity and generalisation.

6.15 Problem 15 — Validation, Test Error, and Model Selection

Suppose a dataset is divided into:

- a training set;
- a validation set;
- a test set.

A model is first fitted on the training set. Several hyperparameter values are then compared using the validation set, and the configuration with the lowest validation error is selected.

6.15.1 Questions

1. What is the purpose of the training set?
2. What is the purpose of the validation set?
3. What is the purpose of the test set?
4. Why should the test set not be repeatedly consulted during model selection?
5. Suppose the test set is used several times to tune hyperparameters. Is it still a truly independent test set?
6. What is the difference between **model selection** and **model evaluation**?

6.15.2 Solution

The **training set** is used to estimate the model parameters.

If the model is represented as

$$f_\theta(\mathbf{x}),$$

then the training process determines

$$\hat{\theta}.$$

The **validation set** is used to make modelling decisions that are not directly estimated as ordinary model parameters.

These decisions may include:

- choosing a model class;
- selecting hyperparameters;
- deciding how much regularisation to apply;
- determining tree depth;
- selecting a subset of predictors;
- comparing alternative feature representations.

Thus, validation data help with **model selection**.

The **test set** serves a different purpose. It is intended to provide an independent estimate of how well the final selected modelling procedure performs on previously unseen data.

The distinction is important.

Suppose we try ten values of a hyperparameter λ and repeatedly inspect test performance until we choose the value that performs best.

The test observations have now influenced the modelling decision.

The test set is no longer functioning purely as independent evaluation data.

Repeated use of the test set can therefore produce an overly optimistic estimate of future performance.

In effect, information from the test set has leaked into the model-development process.

Thus:

- **Model selection** concerns choosing among models or configurations.
- **Model evaluation** concerns estimating the performance of the final selected procedure on genuinely unseen data.

A useful conceptual workflow is

$$\begin{array}{c} \text{Training Data} \longrightarrow \text{Fit Candidate Models} \longrightarrow \text{Validation} \\ \Downarrow \\ \text{Select Final Modelling Procedure} \longrightarrow \text{Independent Test Evaluation.} \end{array}$$

The test set should therefore evaluate the result of the modelling process rather than participate in constructing it.

6.16 Problem 16 — K-Fold Cross-Validation

A regression model is evaluated using five-fold cross-validation. The validation MSE obtained from the five folds is

$$4.2, \quad 5.1, \quad 4.8, \quad 6.0, \quad 4.9.$$

6.16.1 Questions

1. Calculate the five-fold cross-validation estimate of prediction error.
2. What is the role of each fold?
3. How many times is each observation used for validation?
4. How many times is each observation used for training?
5. Why can cross-validation be preferable to a single validation split?

6. Does cross-validation produce five final models for deployment?

6.16.2 Solution

For $K = 5$, the cross-validation estimate is

$$\mathrm{CV}_5 = \frac{1}{5}\sum_{k=1}^{5} \mathrm{MSE}^{(k)}.$$

Substituting the observed fold errors,

$$\begin{aligned}\mathrm{CV}_5 &= \frac{4.2 + 5.1 + 4.8 + 6.0 + 4.9}{5} \\ &= \frac{25.0}{5} \\ &= 5.0.\end{aligned}$$

Thus, the estimated cross-validation error is

$$\mathrm{CV}_5 = 5.0.$$

In five-fold cross-validation, the data are partitioned into five approximately equal subsets.

At each iteration:

- one fold is used as the validation set;
- the other four folds are used for training.

Each fold serves as the validation set exactly once.

Therefore, each observation is used:

- once for validation;
- four times for training.

Cross-validation can be more reliable than a single validation split because the estimate does not depend on only one arbitrary partition of the data.

A single split may produce an unusually easy or difficult validation set. Cross-validation averages performance over several different partitions and therefore provides a more stable assessment.

However, the five fitted models are generally **not** the final deployed models.

They are temporary fits used to estimate predictive performance or to support model selection.

After the modelling procedure has been chosen, a final model is typically refitted using the available training data, according to the selected configuration.

6.17 Problem 17 — Choosing Between K-Fold CV and LOOCV

Suppose a dataset contains $N = 1{,}000$ observations.

Two evaluation procedures are considered:

1. 10-fold cross-validation;
2. Leave-One-Out Cross-Validation.

6.17.1 Questions

1. How many model fits are required for each method?
2. How many observations are used for training in each fit?
3. Which method is computationally more expensive?
4. Why does LOOCV not automatically provide a better estimate simply because it uses more training data in each fit?
5. Why are values such as $K = 5$ or $K = 10$ commonly used in practice?

6.17.2 Solution

For 10-fold cross-validation, the model is fitted

$$10$$

times.

Each fit uses approximately

$$\frac{9}{10} \times 1000 = 900$$

observations for training.

For LOOCV,

$$K = N = 1000,$$

so the model is fitted

$$1000$$

times.

Each fit uses

$$999$$

observations for training and one observation for validation.

LOOCV is therefore much more computationally expensive if the learning procedure must be refitted from scratch at every iteration.

Its advantage is that each fitted model uses almost the full dataset.

However, this does not imply that LOOCV is always superior.

The training datasets in LOOCV differ from one another by only a single observation. They are therefore highly similar, and the resulting error estimates can be strongly correlated.

This can produce relatively high variance in the final estimate.

By contrast, moderate K values such as 5 or 10 often provide a useful practical balance among:

- computational cost;
- size of the training subset;
- stability of the performance estimate.

There is no universally optimal value of K.

The appropriate choice depends on the amount of data, computational constraints, model stability, and the purpose of the evaluation.

6.18 Problem 18 — Bootstrap Sampling and Standard Error

Suppose the original dataset contains four observations:

$$\mathcal{D} = (d_1, d_2, d_3, d_4).$$

Three bootstrap samples are generated:

$$\mathcal{D}^{*1} = (d_1, d_1, d_3, d_4),$$

$$\mathcal{D}^{*2} = (d_2, d_2, d_3, d_3),$$

and

$$\mathcal{D}^{*3} = (d_1, d_2, d_4, d_4).$$

A quantity θ is estimated from each bootstrap sample, producing

$$\hat{\theta}^{*1} = 2.1, \qquad \hat{\theta}^{*2} = 2.7, \qquad \hat{\theta}^{*3} = 2.4.$$

6.18.1 Questions

1. Why can the same observation appear more than once in a bootstrap sample?
2. Why can some observations be absent from a bootstrap sample?
3. Calculate the mean of the bootstrap replicates.
4. Calculate the bootstrap estimate of the standard error.
5. What does this standard error measure conceptually?
6. Is bootstrap primarily intended to estimate predictive performance in the same way as cross-validation?

6.18.2 Solution

Bootstrap sampling is performed **with replacement**.

After an observation is selected, it remains available to be selected again.

This is why a bootstrap sample may contain repeated observations.

For the same reason, some original observations may not be selected at all.

The mean of the bootstrap replicates is

$$\bar{\hat{\theta}}^{*} = \frac{2.1 + 2.7 + 2.4}{3}.$$

Therefore,

$$\bar{\hat{\theta}}^{*} = \frac{7.2}{3} = 2.4.$$

The bootstrap standard error is

$$\widehat{\mathrm{SE}}_{\mathrm{boot}}(\hat{\theta}) = \sqrt{\frac{1}{B-1}\sum_{b=1}^{B}\left(\hat{\theta}^{*b} - \bar{\hat{\theta}}^{*}\right)^2}.$$

Here,

$$B = 3.$$

Thus,

$$\begin{aligned}
\widehat{\mathrm{SE}}_{\mathrm{boot}}(\hat{\theta}) &= \sqrt{\frac{1}{2}\left[(2.1-2.4)^2 + (2.7-2.4)^2 + (2.4-2.4)^2\right]} \\
&= \sqrt{\frac{1}{2}\left[(-0.3)^2 + (0.3)^2 + 0^2\right]} \\
&= \sqrt{\frac{1}{2}\left[0.09 + 0.09\right]} \\
&= \sqrt{0.09} \\
&= 0.3.
\end{aligned}$$

Therefore,

$$\widehat{\mathrm{SE}}_{\mathrm{boot}}(\hat{\theta}) = 0.3.$$

Conceptually, this quantity estimates how much the estimator $\hat{\theta}$ would vary across repeated samples from the underlying population.

The bootstrap attempts to approximate this repeated-sampling behaviour using resamples of the observed dataset.

Its main role is therefore different from that of cross-validation.

Cross-validation is primarily used to estimate **out-of-sample predictive performance** and support model selection.

Bootstrap is primarily used to estimate **sampling variability, uncertainty, and stability**.

6.19 Problem 19 — Data Leakage

A company wants to predict whether a customer will cancel a subscription during the next 30 days.

The dataset contains the following predictors:

- customer age;
- account tenure;
- average monthly usage;
- number of support calls during the previous 90 days;
- number of cancellation-related emails sent during the 30 days after the prediction date.

6.19.1 Questions

1. Which predictor is problematic?
2. What is the problem called?
3. Why might the model appear to perform extremely well during evaluation?
4. Why would the same model perform poorly in production?
5. Give two other examples of data leakage.
6. How can leakage occur during preprocessing even when every predictor seems legitimate?

6.19.2 Solution

The problematic predictor is

> **number of cancellation-related emails sent during the 30 days after the prediction date.**

At prediction time, this information would not yet be available.

It contains information from the future relative to the moment at which the model is supposed to make its prediction.

This is a form of **data leakage**, specifically temporal or target-related leakage.

The model may perform extremely well during development because the leaked predictor contains information strongly associated with the outcome.

For example, customers who cancel may receive or send cancellation-related messages after the prediction date.

The model can exploit this information and appear highly accurate.

However, when the model is deployed, the future information does not exist yet.

Therefore, the relationship used during training is unavailable in the real prediction setting.

The resulting production performance can deteriorate sharply.

Other examples of leakage include:

1. Using a variable created after the outcome occurred.
2. Including a direct transformation of the target among the predictors.
3. Computing preprocessing statistics using the full dataset before splitting into training and validation data.
4. Performing feature selection using information from the validation or test set.
5. Allowing observations from the same entity or time period to appear in both training and validation sets when this violates the intended deployment setting.

Preprocessing leakage is particularly subtle.

Suppose we standardise a predictor using

$$z_i = \frac{x_i - \bar{x}}{s_x}.$$

If $\bar{x}$ and s_x are calculated using the complete dataset before cross-validation, then information from the validation folds has influenced the transformation applied to the training folds.

The correct procedure is to estimate preprocessing quantities using only the training portion within each fold and then apply the fitted transformation to the corresponding validation observations.

A useful principle is:

> **Every operation that learns something from the data must respect the train-validation boundary.**

6.20 Problem 20 — Prediction, Causality, and Distribution Shift

A company trains a model to predict employee turnover.

One of the strongest predictors is the number of meetings an employee attends per week.

The model finds that employees attending more meetings tend to have a lower predicted probability of leaving.

A manager concludes:

We should increase the number of meetings for every employee because the model shows that meetings reduce turnover.

6.20.1 Questions

1. Is this conclusion justified by the predictive model?
2. What has the model actually established?
3. Give at least two alternative explanations for the observed association.
4. What additional information or methodology would be needed to support a causal conclusion?
5. Suppose the model performs well on historical data but the company later adopts fully remote work and substantially changes its meeting practices. Why might performance deteriorate?
6. What is the distinction between distribution shift and concept drift?

6.20.2 Solution

The manager's conclusion is **not justified by the predictive model alone**.

The model has identified an association between meeting attendance and employee turnover that is useful for prediction.

It has not established that increasing meeting attendance would cause turnover to decrease.

The predictive result supports a statement of the form

> employees with certain meeting patterns tend to have different turnover probabilities.

It does not automatically support the intervention statement

> forcing employees to attend more meetings will reduce turnover.

Several alternative explanations are possible.

For example:

- employees in more senior or stable roles may naturally attend more meetings and also have lower turnover;
- employees already planning to leave may disengage and attend fewer meetings;
- team structure, role type, or management quality may influence both meeting frequency and turnover;
- meeting attendance may be a proxy for another variable rather than the causal mechanism itself.

The distinction can be written conceptually as

$$P(Y \mid X)$$

versus a causal quantity such as

$$P(Y \mid \text{do}(X = x)).$$

A predictive model generally estimates relationships of the first kind.

Supporting a causal conclusion may require additional assumptions and methods, such as:

- randomised experiments;
- quasi-experimental designs;
- causal graphical models;
- instrumental variables;
- matching or weighting methods;
- carefully justified causal identification assumptions.

The second part of the problem concerns deployment over time.

Suppose the model was trained under an environment in which most employees worked in offices, but the company later adopts remote work.

The distribution of predictors may change.

For example, meeting frequency, communication patterns, work location, and team interaction may all differ from those represented in the training data.

This is a form of **distribution shift**.

More generally, distribution shift means that the statistical distribution encountered during deployment differs from the distribution represented during training.

A particularly important form is **concept drift**, where the relationship between predictors and response changes.

For example, historically,

$$P(Y \mid \mathbf{X})$$

may follow one relationship, while after organisational change the same predictor values may correspond to different turnover probabilities.

Thus, concept drift concerns changes in the predictive relationship itself, while distribution shift is the broader term covering changes between training and deployment distributions.

A model can therefore be technically well fitted, carefully cross-validated, and still deteriorate in production if the environment changes substantially.

This illustrates an important limitation of predictive modelling:

> **Generalisation is always relative to the population and conditions represented by the available data.**